

# 머신러닝을 위한 선형대수학

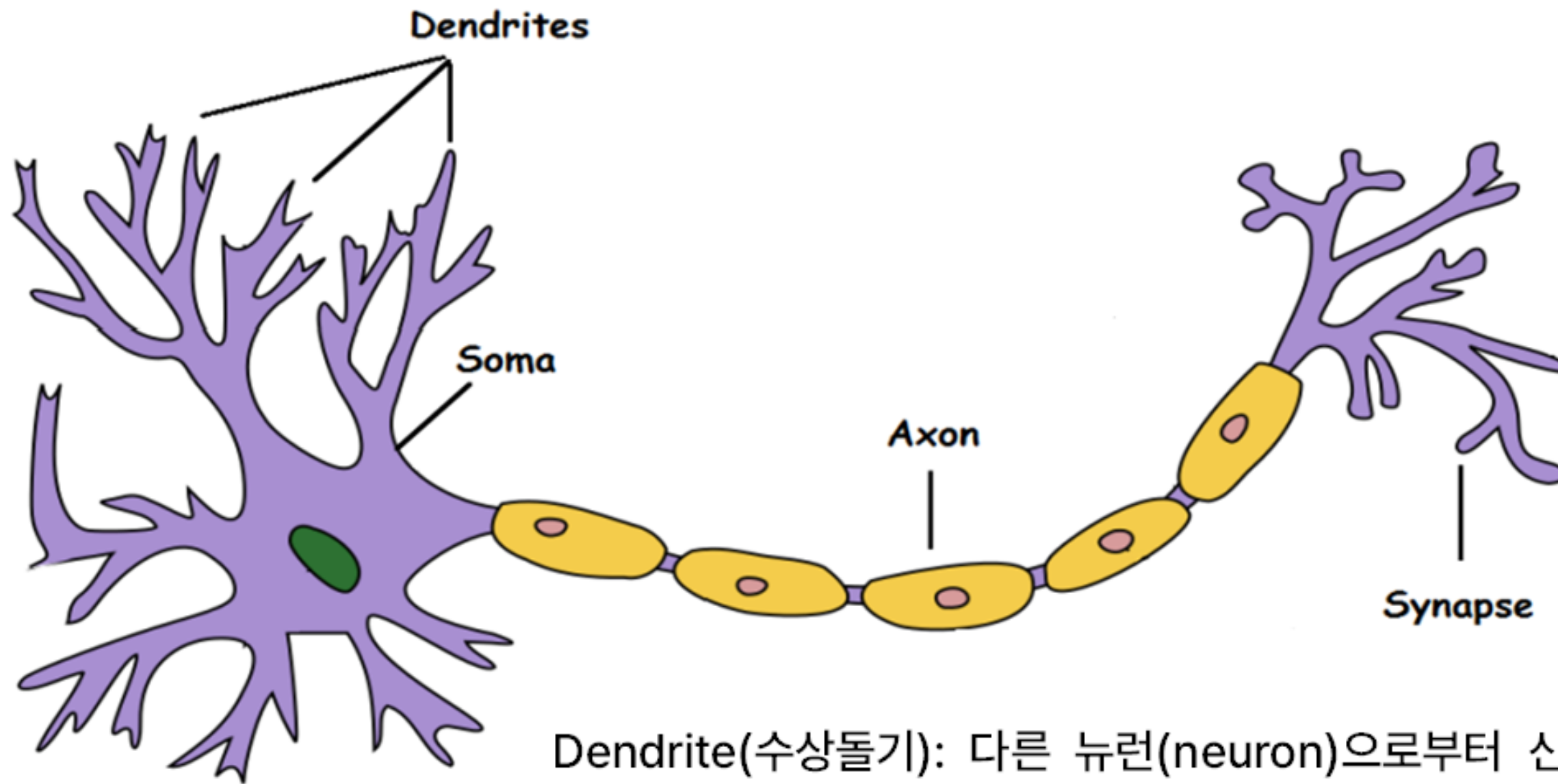
12 신경망의 기본 구조와 행렬 연산

## 목차 12 신경망의 기본 구조와 행렬 연산

---

1. 신경망의 역사
2. 신경망의 수학적 구조
3. 역전파 알고리즘

# 1. 신경망의 역사

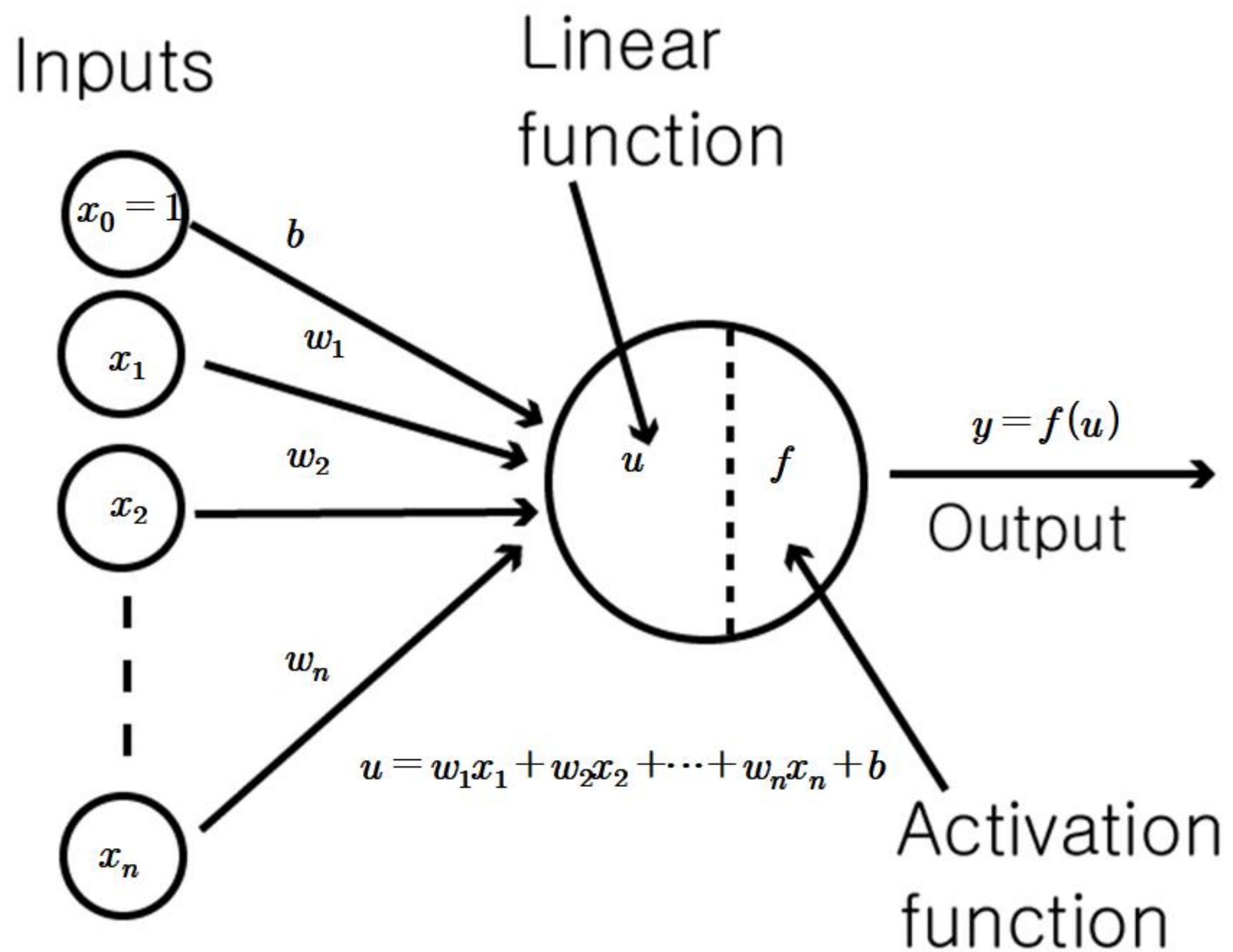


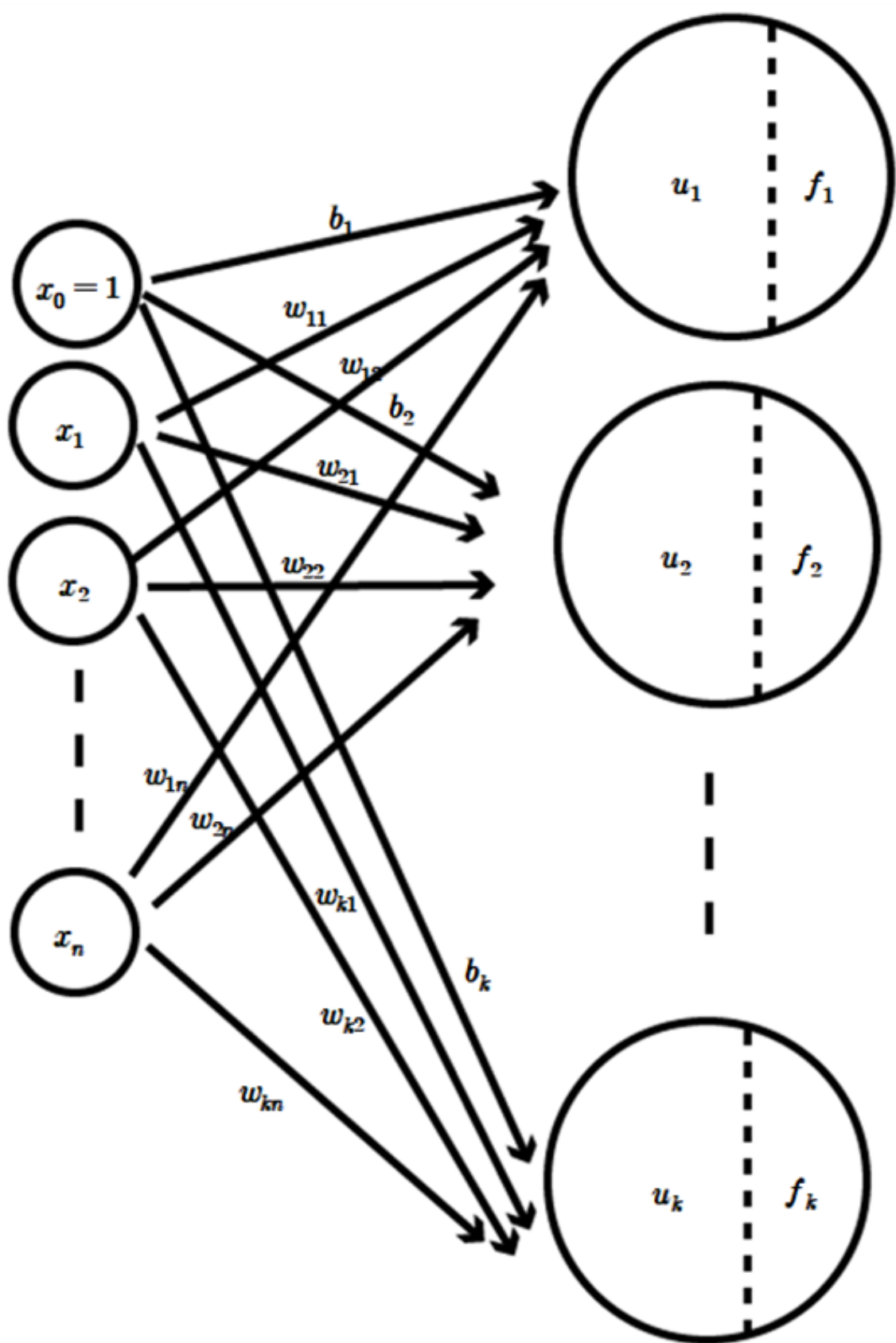
Dendrite(수상돌기): 다른 뉴런(neuron)으로부터 신호를 전달 받음

Soma(체세포): 정보처리 역할

Axon(축색돌기): 뉴런의 출력을 전달

Synapse(시냅스): 다른 뉴런에 연결되는 지점





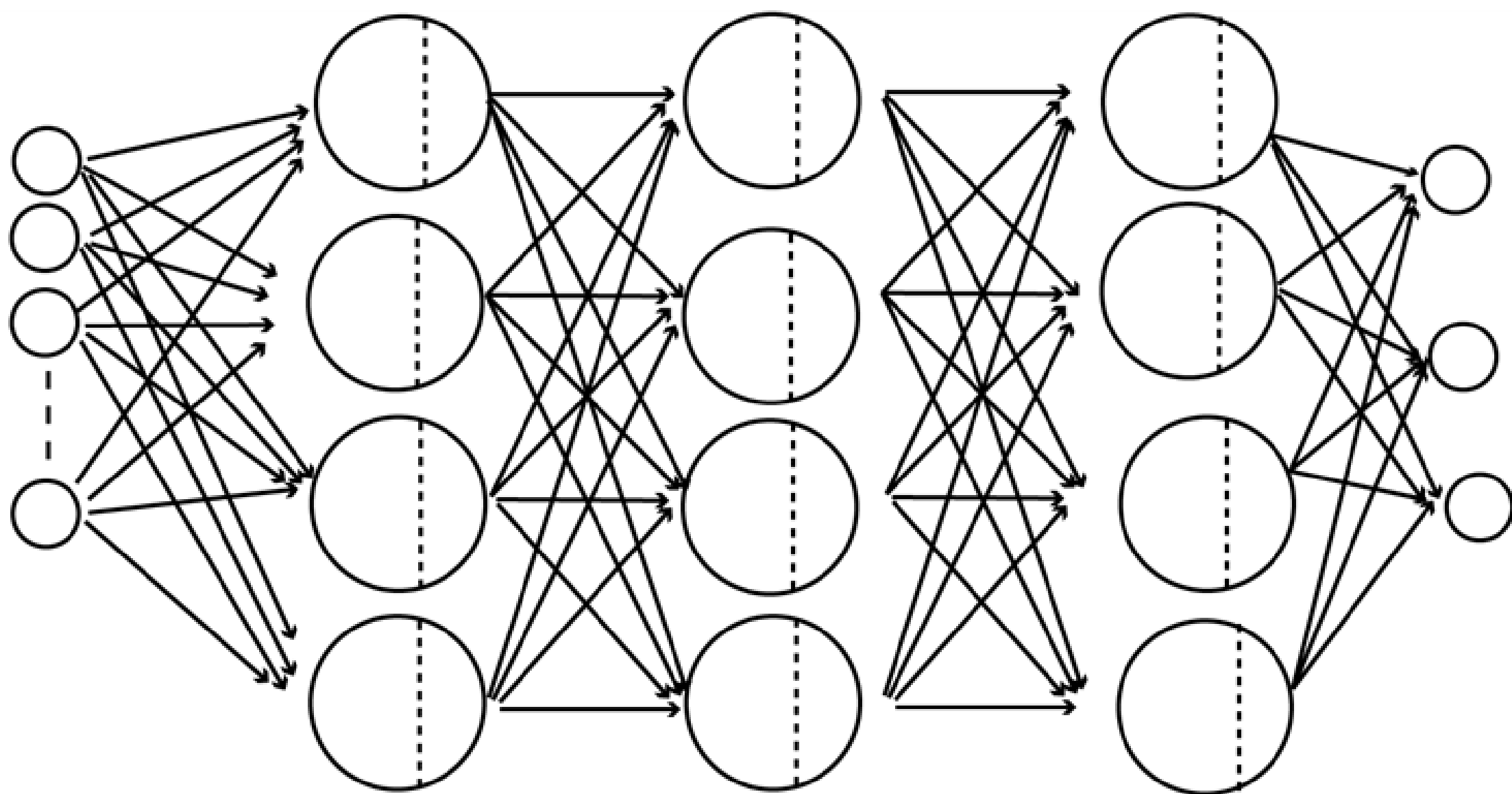
$$u_1 = w_{11}x_1 + w_{12}x_2 + \cdots + w_{1n}x_n + b_1$$

$$u_2 = w_{21}x_1 + w_{22}x_2 + \cdots + w_{2n}x_n + b_2$$

$\vdots$

$$u_k = w_{k1}x_1 + w_{k2}x_2 + \cdots + w_{kn}x_n + b_k$$

$$\begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_k \end{pmatrix} = \begin{pmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \cdots & \vdots \\ w_{k1} & w_{k2} & \cdots & w_{kn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_k \end{pmatrix} \Rightarrow \begin{pmatrix} f(u_1) \\ f(u_2) \\ \vdots \\ f(u_k) \end{pmatrix}$$



$$W_1x + b_1 = u_1 \Rightarrow f(u_1)$$

$$W_2f(u_1) + b_2 = u_2 \Rightarrow f(u_2)$$

$$W_3f(u_2) + b_3 = u_3 \Rightarrow f(u_3)$$

$$W_4f(u_3) + b_4 = u_4 \Rightarrow f(u_4)$$

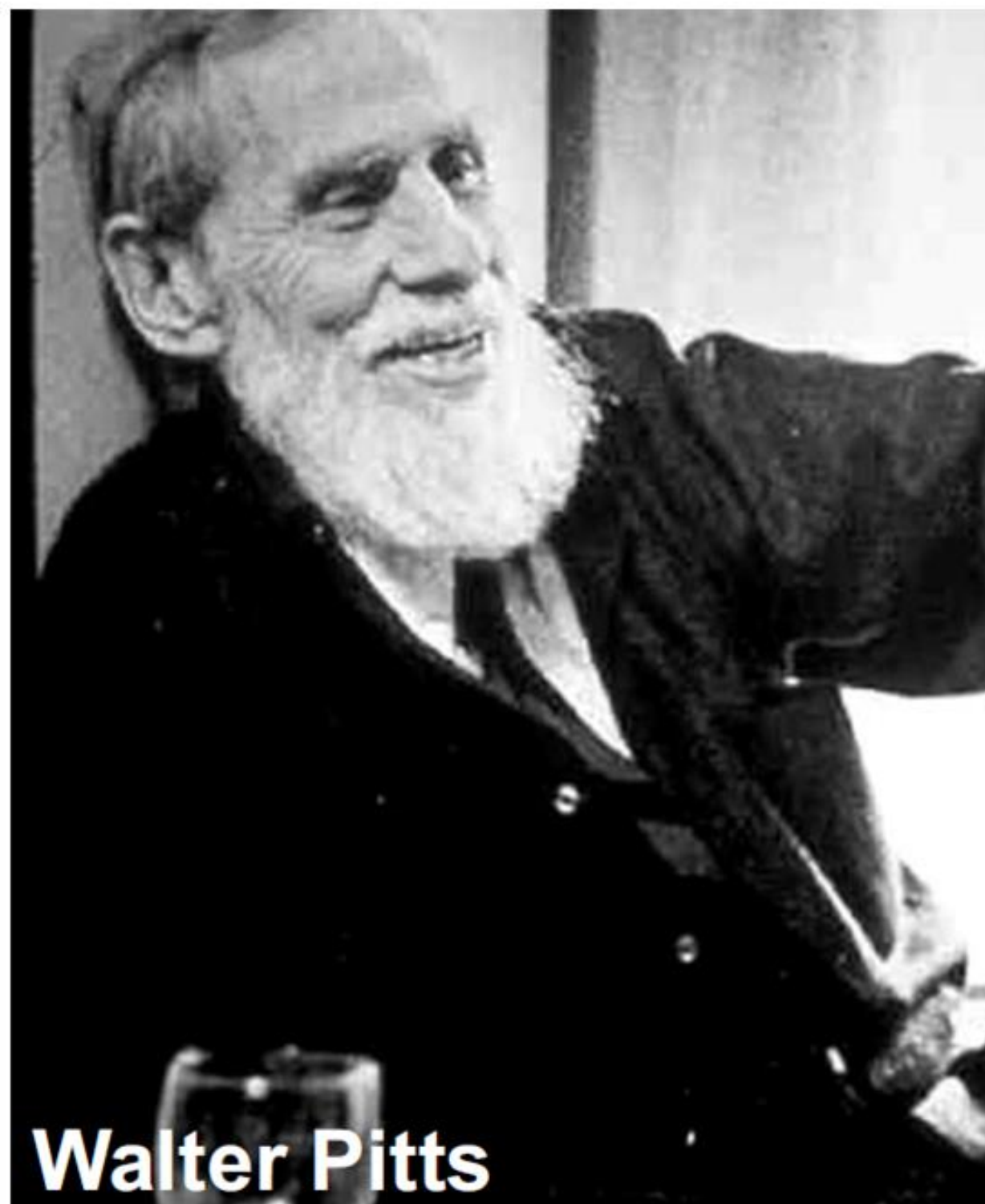
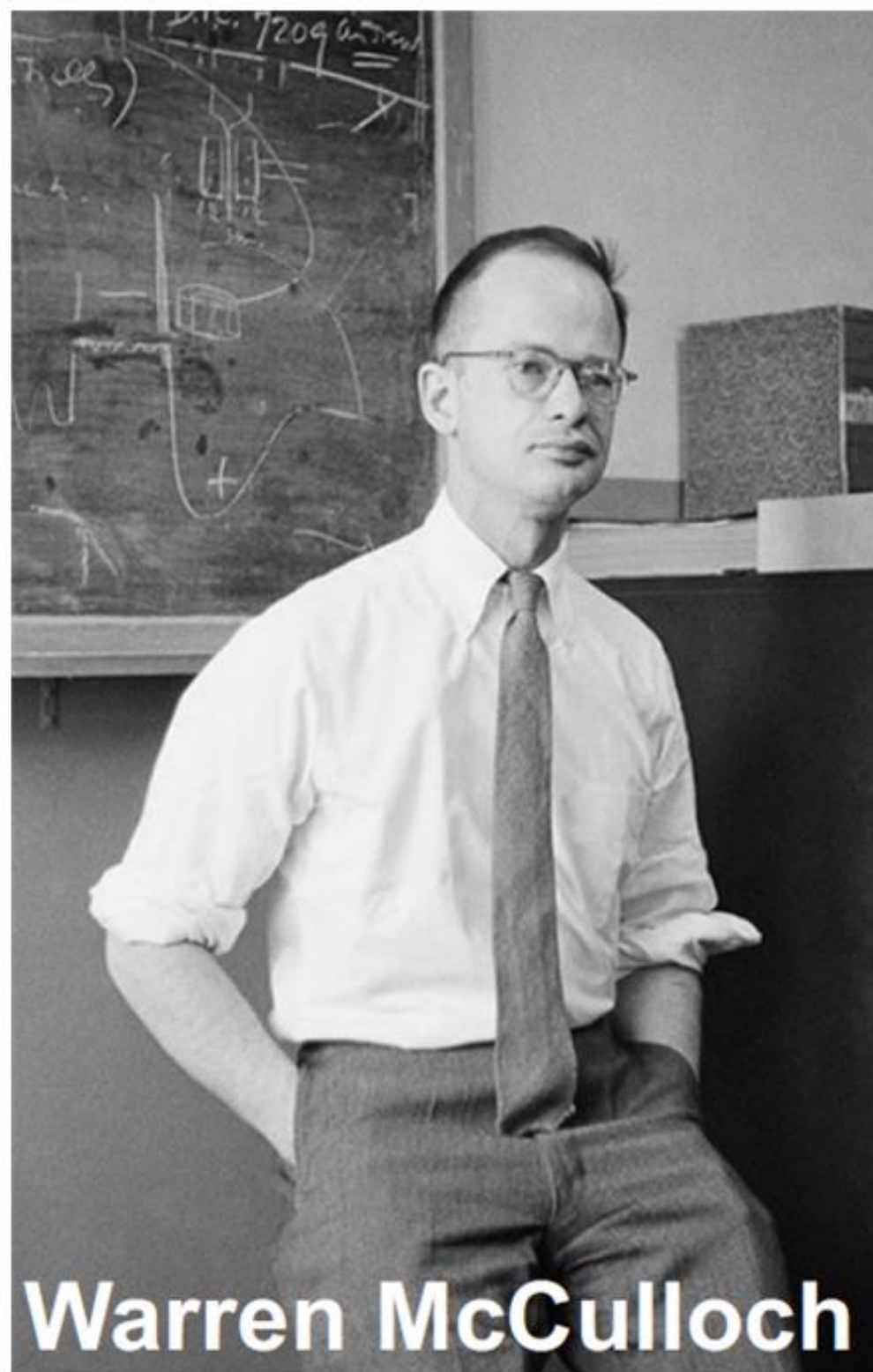


**1943년:** 인공신경망의 시작은 신경생리학자인 Warren McCulloch와 그 당시 젊은 수학자였던 Walter Pitts로보고 있다. 인공신경망이라는 개념을 소개하고, 뇌가 어떻게 논리적, 수학적 연산을 통해 표현될 수 있는지 탐구했다.

Warren McCulloch와 Walter Pitts의 주요 저서인 "신경 활동에 내재된 아이디어의 논리적 계산"[1943]

"The Logical Calculus of the Ideas Immanent in Nervous Activity"[1943]

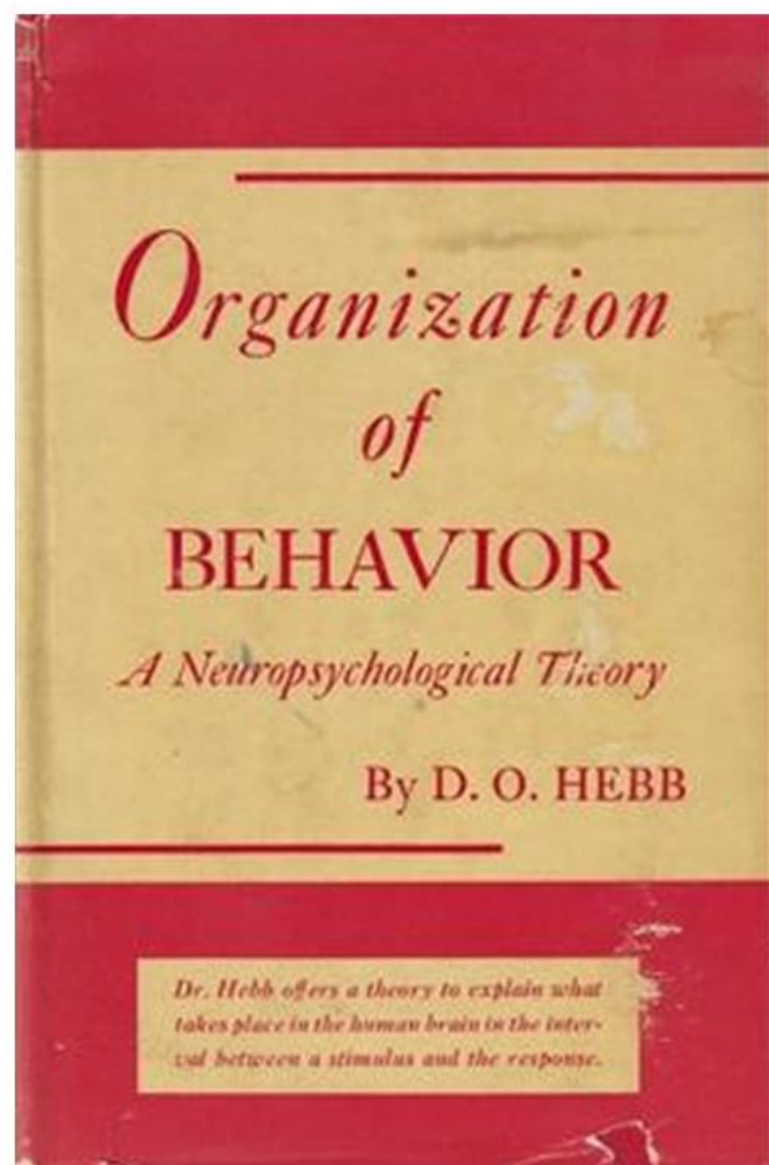
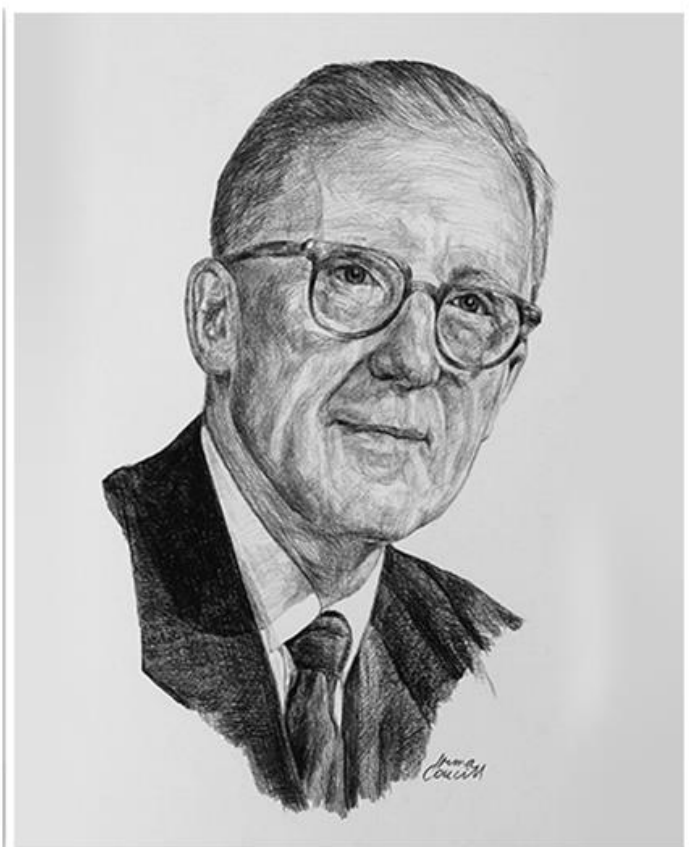
그들의 네트워크는 간단한 요소들을 기반으로 구성 됐는데, 이들은 이진 장치로 간주되고 고정된 임계값을 갖고 있었습니다. 그들 모델의 결과는 신경 활동의 "'all-or-none" " 특성을 가진 간단한 논리 함수였다.





## 1949년: Hebb, D.: The Organization of Behavior. Wiley, New York (1949)

1949년, 신경 심리학자인 헤브(Hebb)가 '행동의 조직'이라는 책에서, 신경 회로가 사용될 때마다 강화된다고 지적했는데 이는 인간이 학습하는 방식의 근본적인 개념입니다.



The father of neuropsychology[신경 심리학]

Throughout his life, Dr. Donald O. Hebb maintained a deep interest in the relationship between mind and brain. In 1949, he published his classic work *The Organization of Behaviour*, which served as a bridge between psychology and neuroscience. In it, Hebb theorized that cognitive processing was the result of connections between neuron assemblies. The book had an immediate impact and led Hebb to become known as "The Father of Neuropsychology." Throughout his distinguished career, Hebb occupied many important offices in North American psychological associations, was a Fellow of the Royal Society of London, and was the recipient of over a dozen honorary degrees.

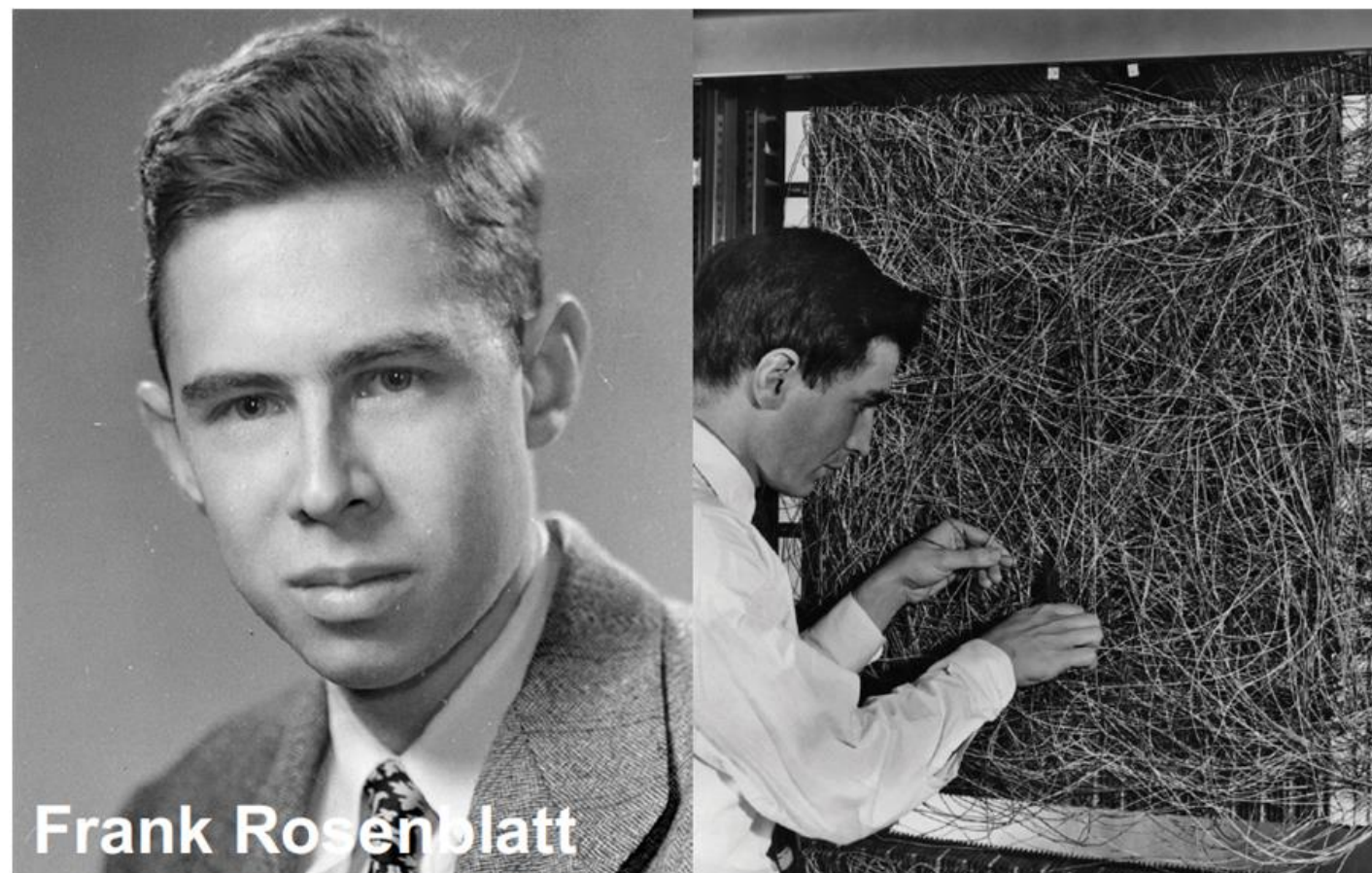


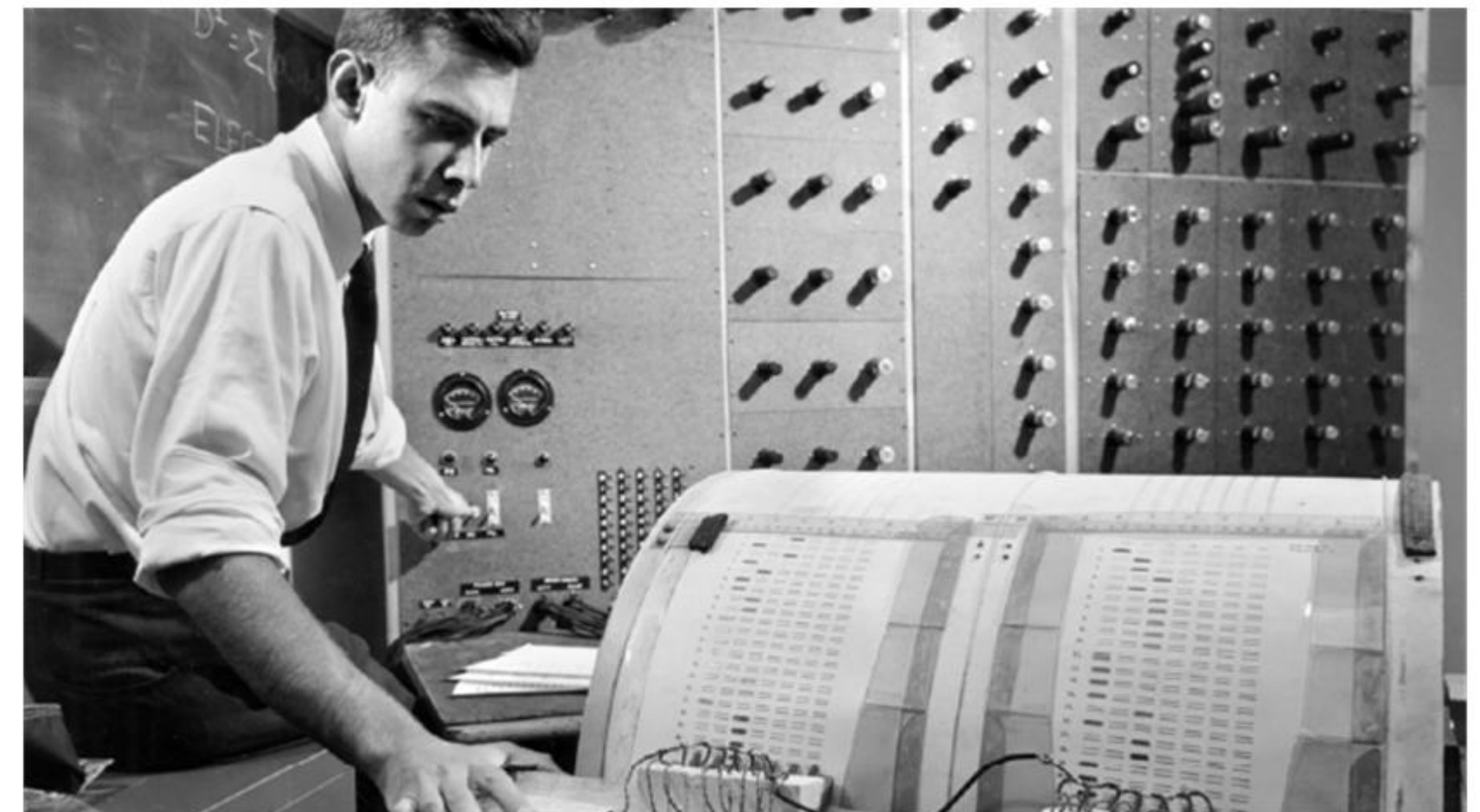
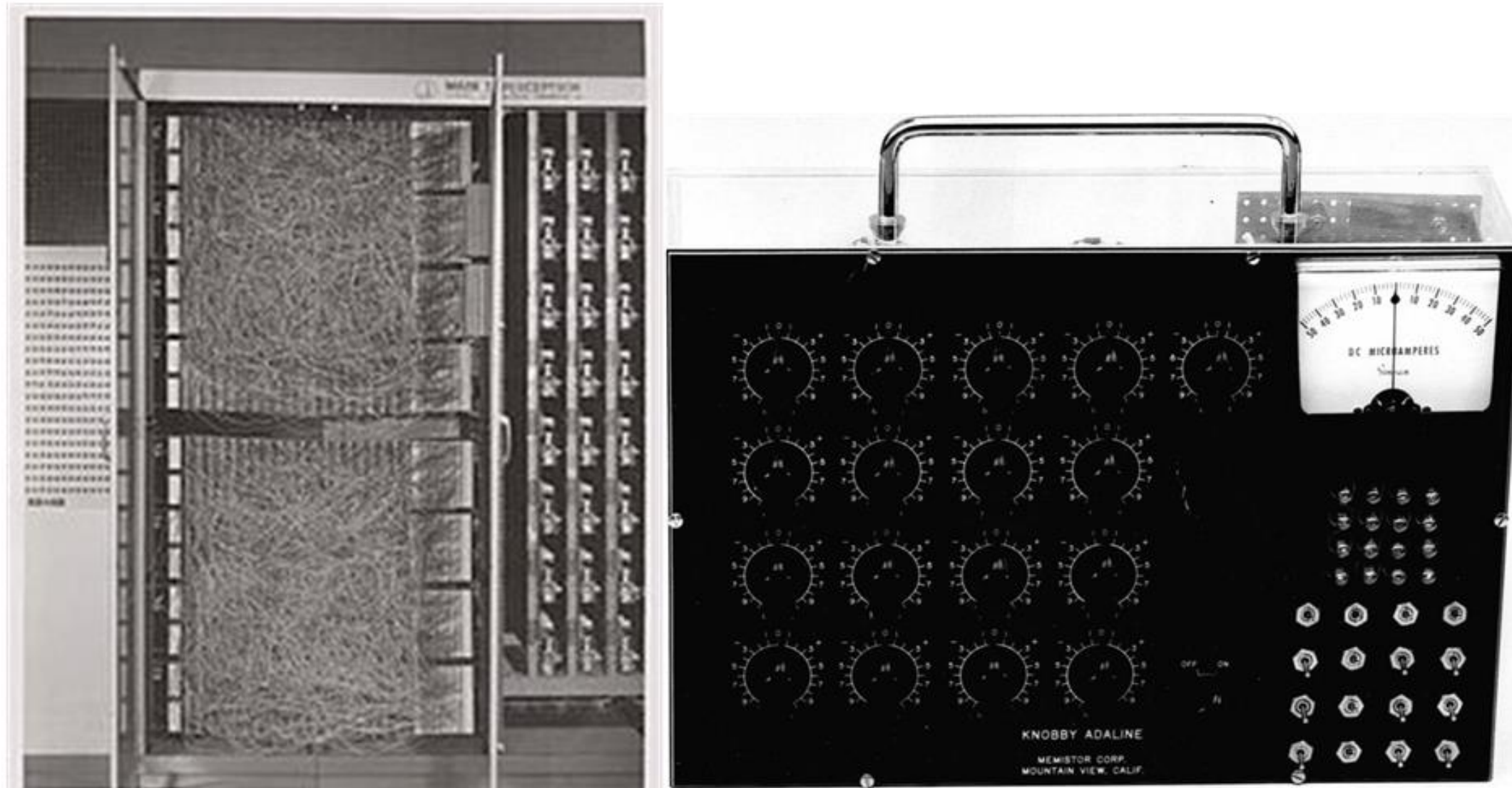
**1958년:** 1958년 심리학자인 Frank Rosenblatt는 perceptron을 발명하였다.

퍼셉트론은 생물학적 원리에 따라 구축된 전자 장치인데 학습 능력이 있음은 보였다.

주로 하드웨어적인 접근을 하였다.

Rosenblatt, F.: The perceptron: a probabilistic model for information storage and organization in the brain. Psychol. Rev. 65(6), 386–408 (1958)







## NEW NAVY DEVICE LEARNS BY DOING

Psychologist Shows Embryo  
of Computer Designed to  
Read and Grow Wiser

WASHINGTON, July 7 (UPI)  
—The Navy revealed the embryo of an electronic computer today that it expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence.

The embryo—the Weather Bureau's \$2,000,000 "704" computer—learned to differentiate between right and left after fifty attempts in the Navy's demonstration for newsmen.

The service said it would use this principle to build the first of its Perceptron thinking machines that will be able to read and write. It is expected to be finished in about a year at a cost of \$100,000.

Dr. Frank Rosenblatt, designer of the Perceptron, conducted the demonstration. He said the machine would be the first device to think as the human brain. As do human beings,

Perceptron will make mistakes at first, but will grow wiser as it gains experience, he said.

Dr. Rosenblatt, a research psychologist at the Cornell Aeronautical Laboratory, Buffalo, said Perceptrons might be fired to the planets as mechanical space explorers.

### Without Human Controls

The Navy said the perceptron would be the first non-living mechanism "capable of receiving, recognizing and identifying its surroundings without any human training or control."

The "brain" is designed to remember images and information it has perceived itself. Ordinary computers remember only what is fed into them on punch cards or magnetic tape.

Later Perceptrons will be able to recognize people and call out their names and instantly translate speech in one language to speech or writing in another language, it was predicted.

Mr. Rosenblatt said in principle it would be possible to build brains that could reproduce themselves on an assembly line and which would be conscious of their existence.

## 1958 New York Times...

In today's demonstration, the "704" was fed two cards, one with squares marked on the left side and the other with squares on the right side.

### Learns by Doing

In the first fifty trials, the machine made no distinction between them. It then started registering a "Q" for the left squares and "O" for the right squares.

Dr. Rosenblatt said he could explain why the machine learned only in highly technical terms. But he said the computer had undergone a "self-induced change in the wiring diagram."

The first Perceptron will have about 1,000 electronic "association cells" receiving electrical impulses from an eye-like scanning device with 400 photo-cells. The human brain has 10,000,000,000 responsive cells, including 100,000,000 connections with the eyes.



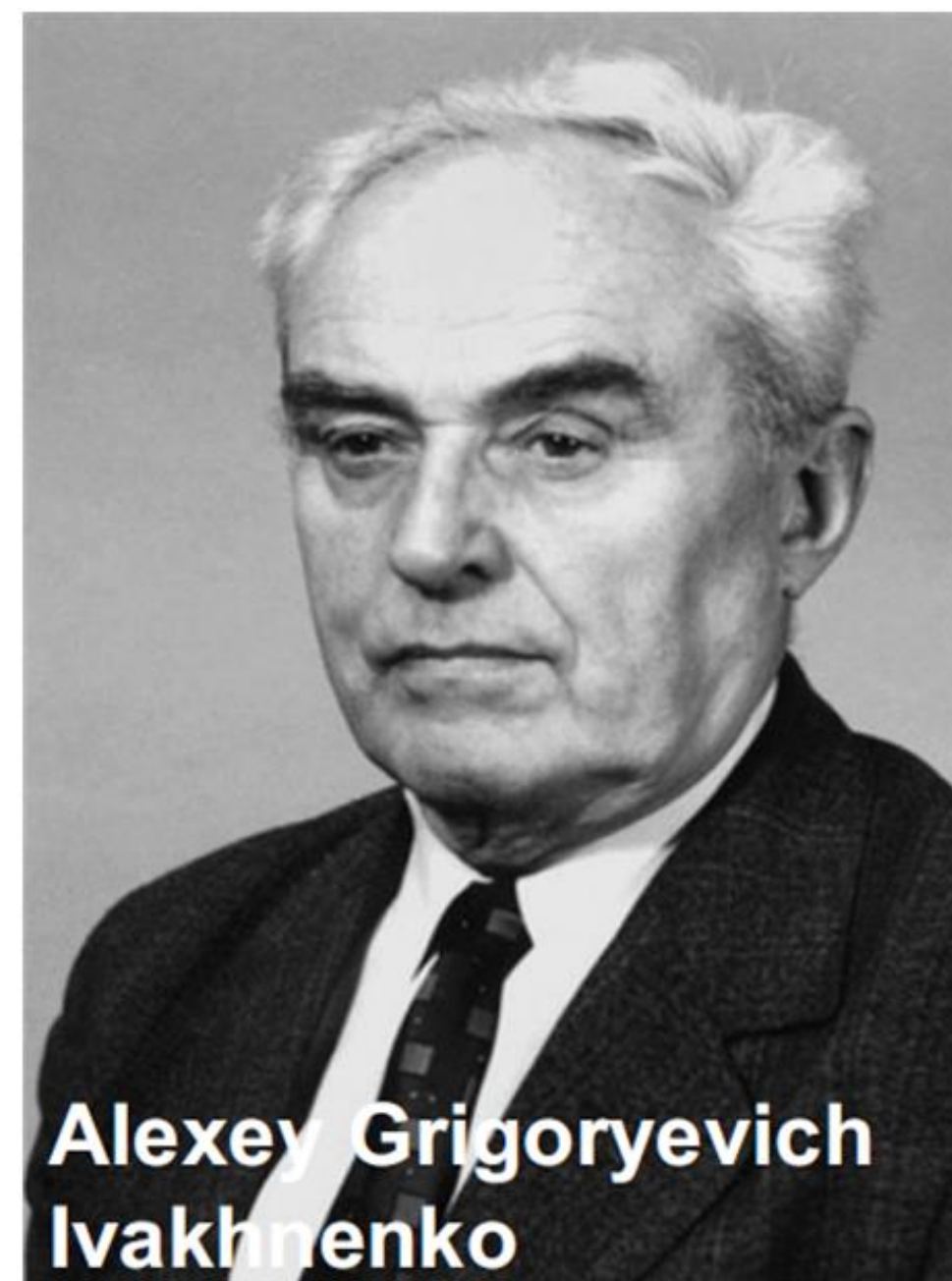
"The Navy revealed the embryo of an electronic computer today that **it expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence** Dr. Frank Rosenblatt, a research psychologist at the Cornell Aeronautical Laboratory, Buffalo, said Perceptrons might be fired to the planets as mechanical space explorers"

The New York Times July 08, 1958



**1965년:** Alexey Ivakhnenko에 의해서 **deep learning network**가 구성되었다.

Ivakhnenko는 GMDH( group method of data handling)를 사용하여 1971년에 8층 심층 신경망을 생성할 수 있었으며, 그는 Alpha라는 컴퓨터 식별 시스템에서 학습 과정을 성공적으로 시연했다고 한다.



**Alexey Grigoryevich  
Ivakhnenko**

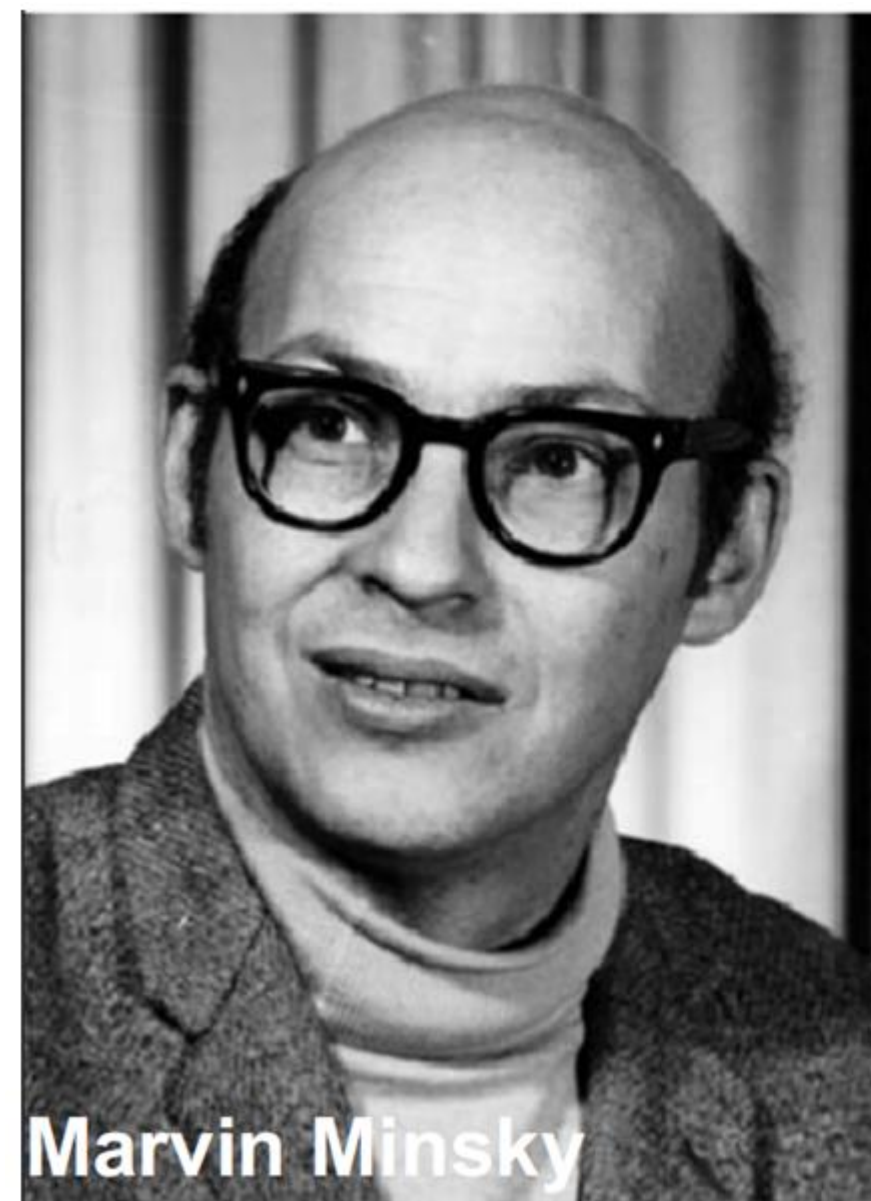


## 1969년[AI Winter]: Marvin Minsky

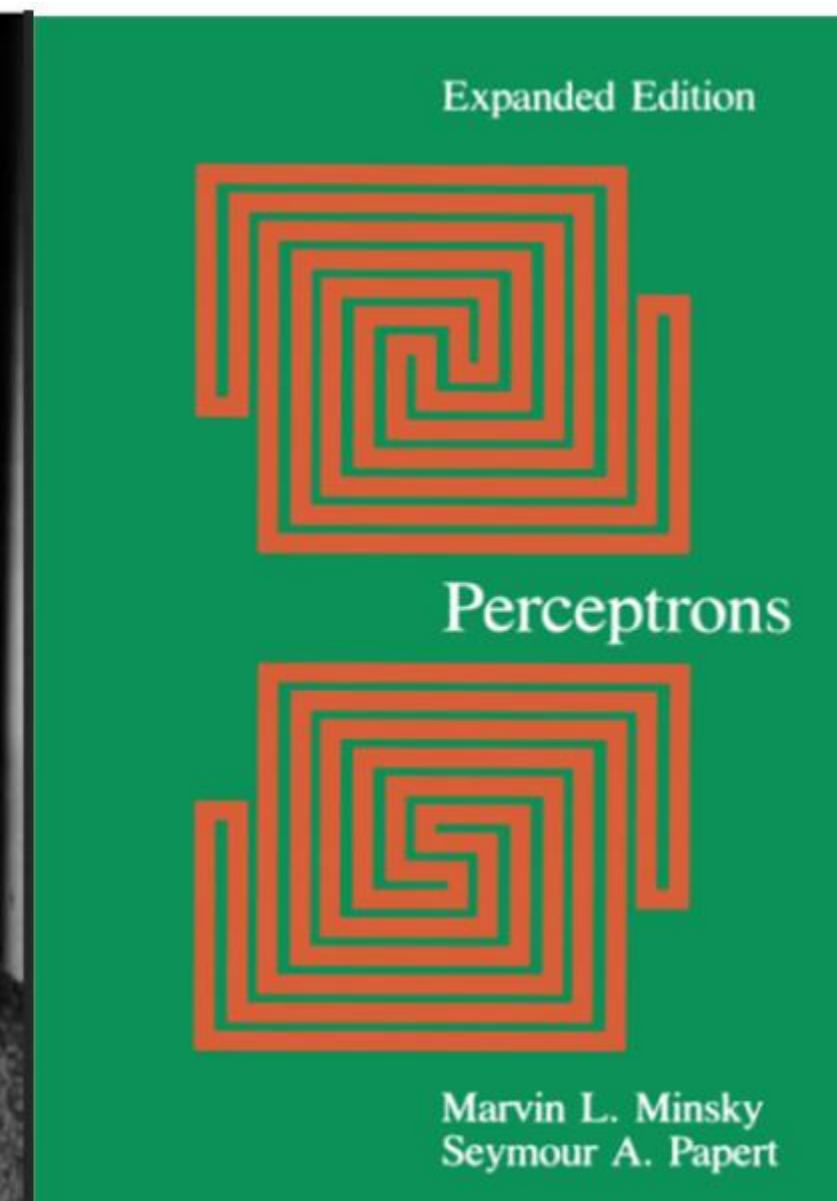
Perceptrons (1969) by Marvin Minsky, founder of the MIT AI Lab

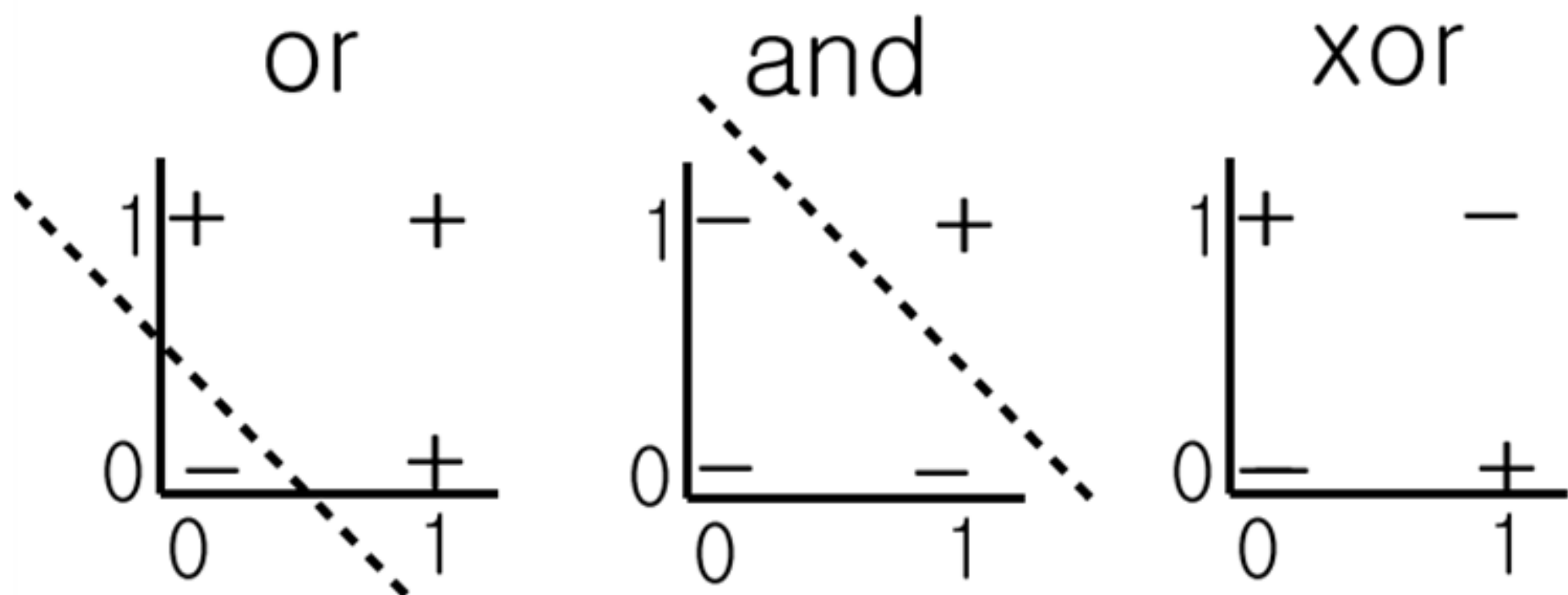
We need to use MLP, multilayer perceptrons (multilayer neural nets)

No one on earth had found a viable way to train MLPs good enough to learn such simple functions.



Marvin Minsky



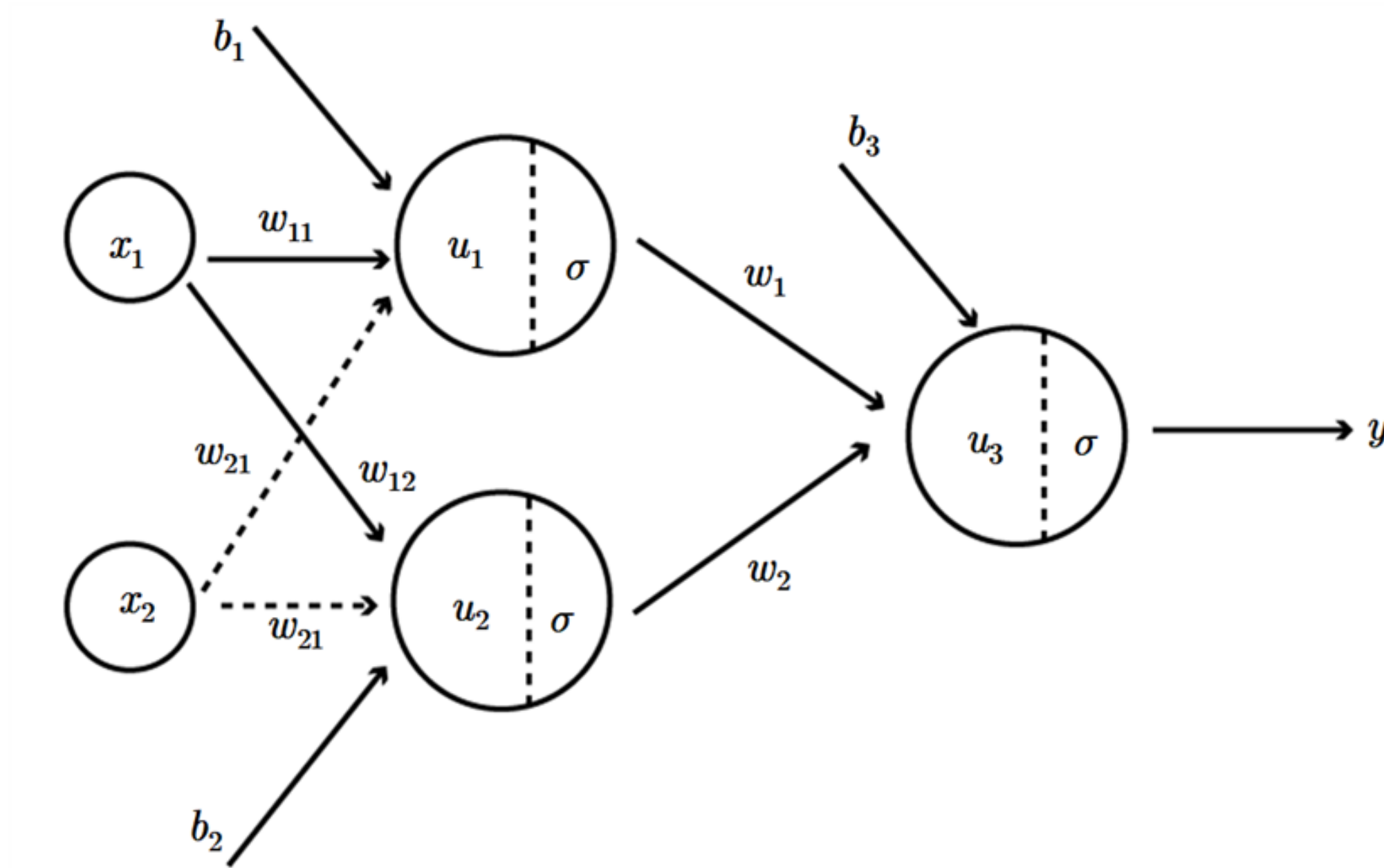


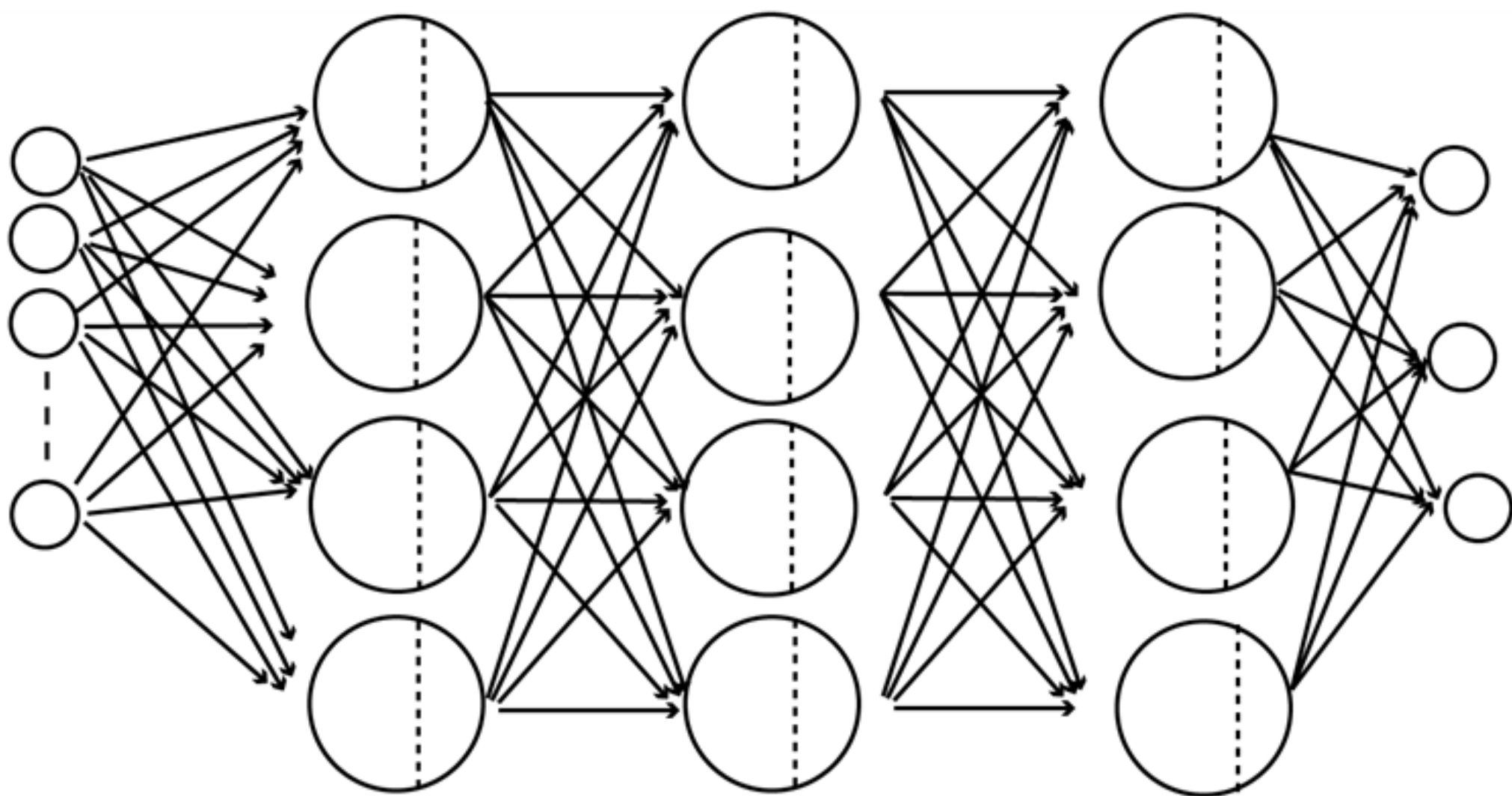
신경망 연구의 일시적 침체: Perceptrons 책에서 단층 퍼셉트론의 한계를 지적한 것이 일부 연구자들 사이에서 신경망 연구에 대한 회의감을 불러일으켰습니다. 이로 인해 신경망 연구가 한동안 침체기(10년 이상)를 겪었습니다.

$XOR$ 문제는  $y = ax + b$  형태의 단층 퍼셉트론으로는 문제를 해결 할 수 없다.



하지만 다음과 같은 신경망에 의해서  $XOR$ 문제는 간단히 해결된다.





$$u_1 = w_{11}x_1 + w_{12}x_2 + \cdots + w_{1n}x_n + b_1$$

$$u_2 = w_{21}x_1 + w_{22}x_2 + \cdots + w_{2n}x_n + b_2$$

$\vdots$

$$u_k = w_{k1}x_1 + w_{k2}x_2 + \cdots + w_{kn}x_n + b_k$$

$$\begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_k \end{pmatrix} = \begin{pmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \cdots & \vdots \\ w_{k1} & w_{k2} & \cdots & w_{kn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_k \end{pmatrix} \Rightarrow \begin{pmatrix} f(u_1) \\ f(u_2) \\ \vdots \\ f(u_k) \end{pmatrix}$$



## Beginning of Backpropagation

(1974, 1982 by Paul Werbos, 1986 by Hinton)

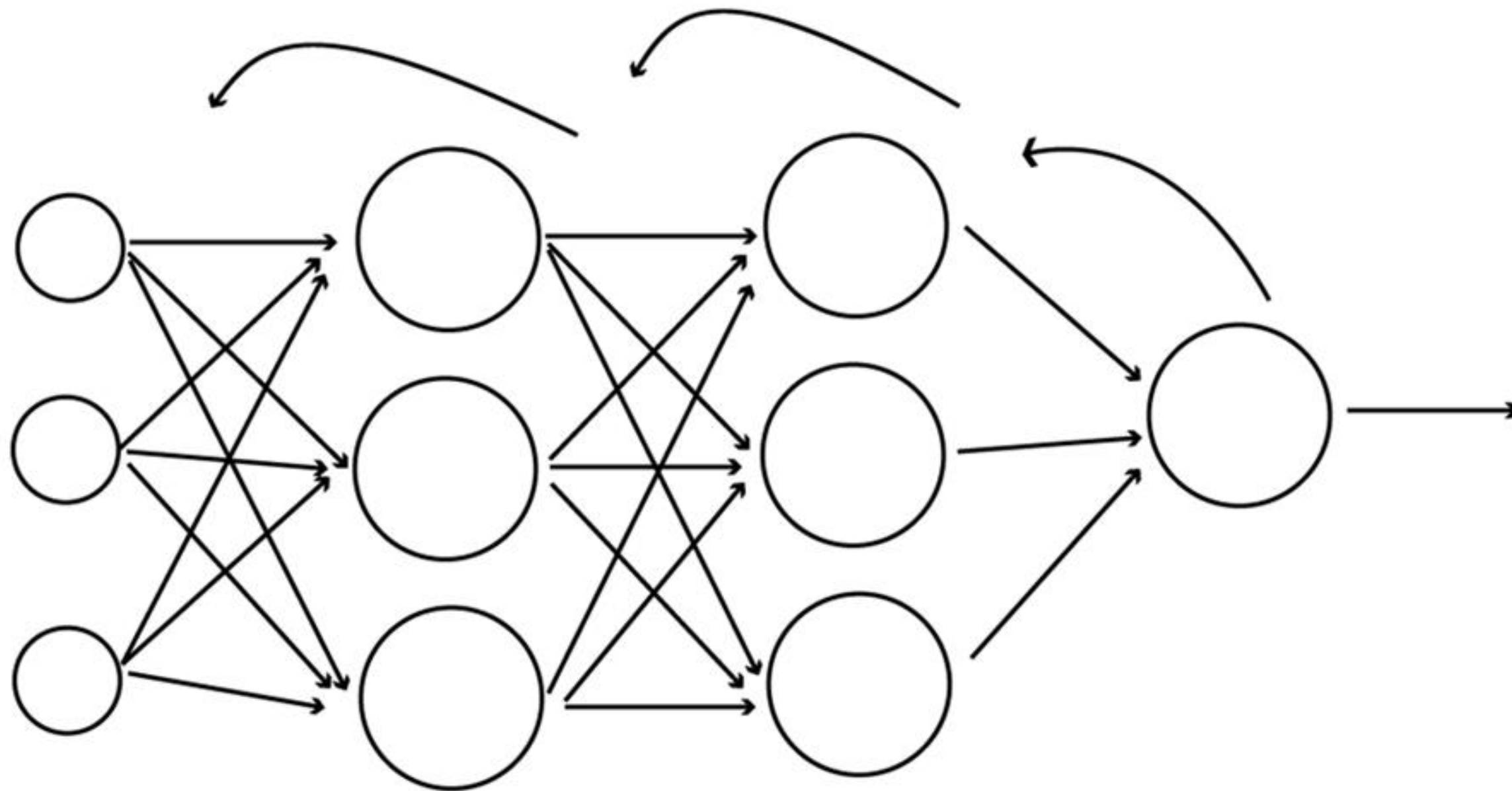
Paul John Werbos (born 1947) is an American social scientist and machine learning pioneer. He is best known for his 1974 dissertation, which first described the process of training artificial neural networks through **backpropagation** of errors. He also was a pioneer of recurrent neural networks.(Wikipedia)



Paul John Werbos



Hinton



[입력층]

[은닉층]

[출력층]

""Adjustments of weights through backpropagation.""



**1980년대:** 역전파(backpropagation) 알고리즘이 소개되면서 다층 퍼셉트론(multilayer perceptron)이라는 다층 구조의 인공신경망이 다시 주목받게 되었습니다. 이로써 비선형 문제를 해결할 수 있는 인공신경망의 잠재력이 크게 증가했습니다.

**1990년대:** 컴퓨터의 성능 향상과 데이터 수집 기술의 발전으로 인공신경망의 연구가 다시 활발해졌습니다. 이때 다양한 신경망 구조와 학습 알고리즘이 제안되었습니다.

**2000년대:** 지능형 시스템과 기계학습 분야의 성장과 함께, 인공신경망의 활용이 다양한 분야로 확대되었습니다. 특히 딥러닝(deep learning)이라는 개념이 부상하여, 다층 신경망의 여러 층을 쌓아올려 더 복잡하고 효과적인 모델을 만드는 연구가 활발해졌습니다.

**2010년대 이후:** 딥러닝이 다양한 분야에서 높은 성능을 발휘하며 기계번역, 음성 인식, 이미지 인식, 게임 인공지능 등 다양한 분야에서 혁신을 이끌었습니다. 또한, GPU와 같은 병렬 컴퓨팅 기술의 발전으로 대용량 데이터를 다루며 신경망을 훈련하는 것이 가능해졌습니다.

이러한 연구와 기술 발전은 현재까지도 계속되고 있으며, 더욱 효율적이고 강력한 인공신경망 모델을 만들기 위한 연구가 이어지고 있습니다.



**딥러닝의 한계점:** 딥러닝 하면 모르는 사람이 없을 만큼 성능을 인정받고 있으나 아직 이론적인 이해는 이를 따라가지 못하고 있다.

예를 들면 층수가 늘어나면 왜 높은 성능이 나오는가?에 대한 납득할 만한 이론적 배경이 아직 없다. 각 과제마다 해보지 않고는 성능을 알 수 없다. 안될 경우 왜 안 되는지 논리적이 설명이 어렵다.

## 2. 신경망의 수학적 구조

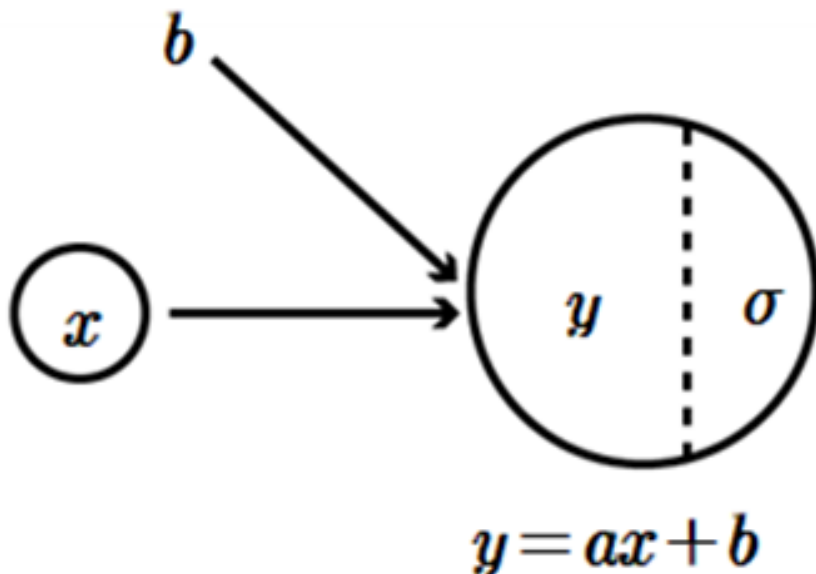


## 1. 뉴런의 수학적 모델

신경망 단위 구조: 입력값이 하나인 경우

신경망의 수학적 구조는 매우 간단한 1차 함수 구조를 하고 있다.

$$y = wx + b, f(x) = wx + b$$



여기서  $x$  는 입력 값이고  $w$  값은 가중치(weight) 값이고  $b$  를 절편 또는 편향(bias)라 부른다. 입력 값  $x$  에 가중치를 곱하고 절편값을 더해서 출력 값  $y$  을 얻게 된다. 같은 의미이지만 편의에 따라서 출력값을  $y$  로 또는  $f(x)$  로 표현하자. 보통은  $y$  로 표현한다.



신경망 구조: 입력값이 여러 개인 경우

입력값이 하나가 아니라 여러 개를 받아서 출력하는 경우이다. 이 경우  $n$  개의 입력값  $x_1, x_2, \dots, x_n$  에 각각에 가중치 값  $w_1, w_2, \dots, w_n$  을 곱하고 절편값  $b$  을 더하면 된다.

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

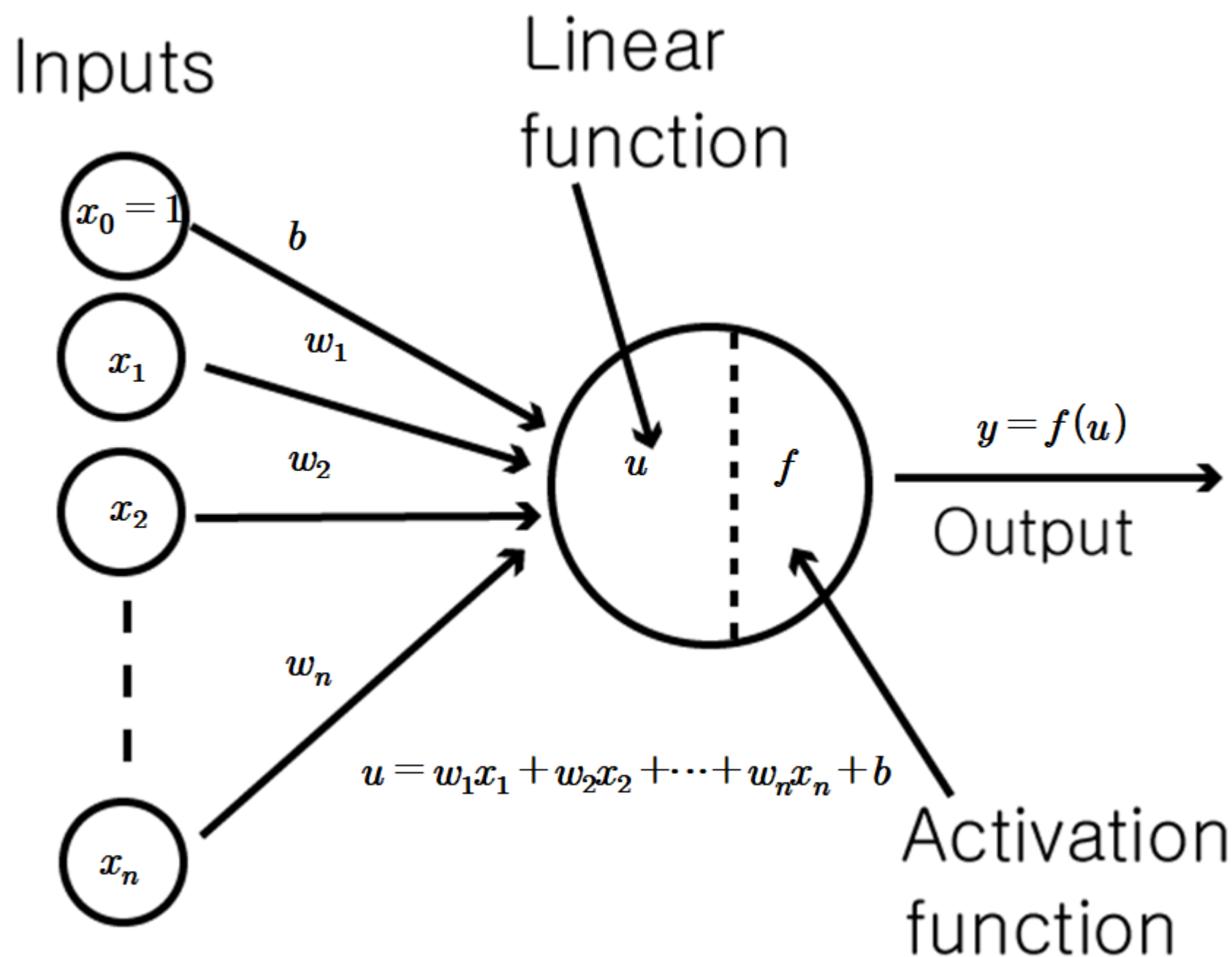
위식은 벡터방식으로 표현하면 간단하다.

$w = (w_1, w_2, \dots, w_n, b)$ ,  $x = (x_1, x_2, \dots, x_n, 1)$ 이라 하면

$$y = w \cdot x^T$$

로 표시 된다.

$$u = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$





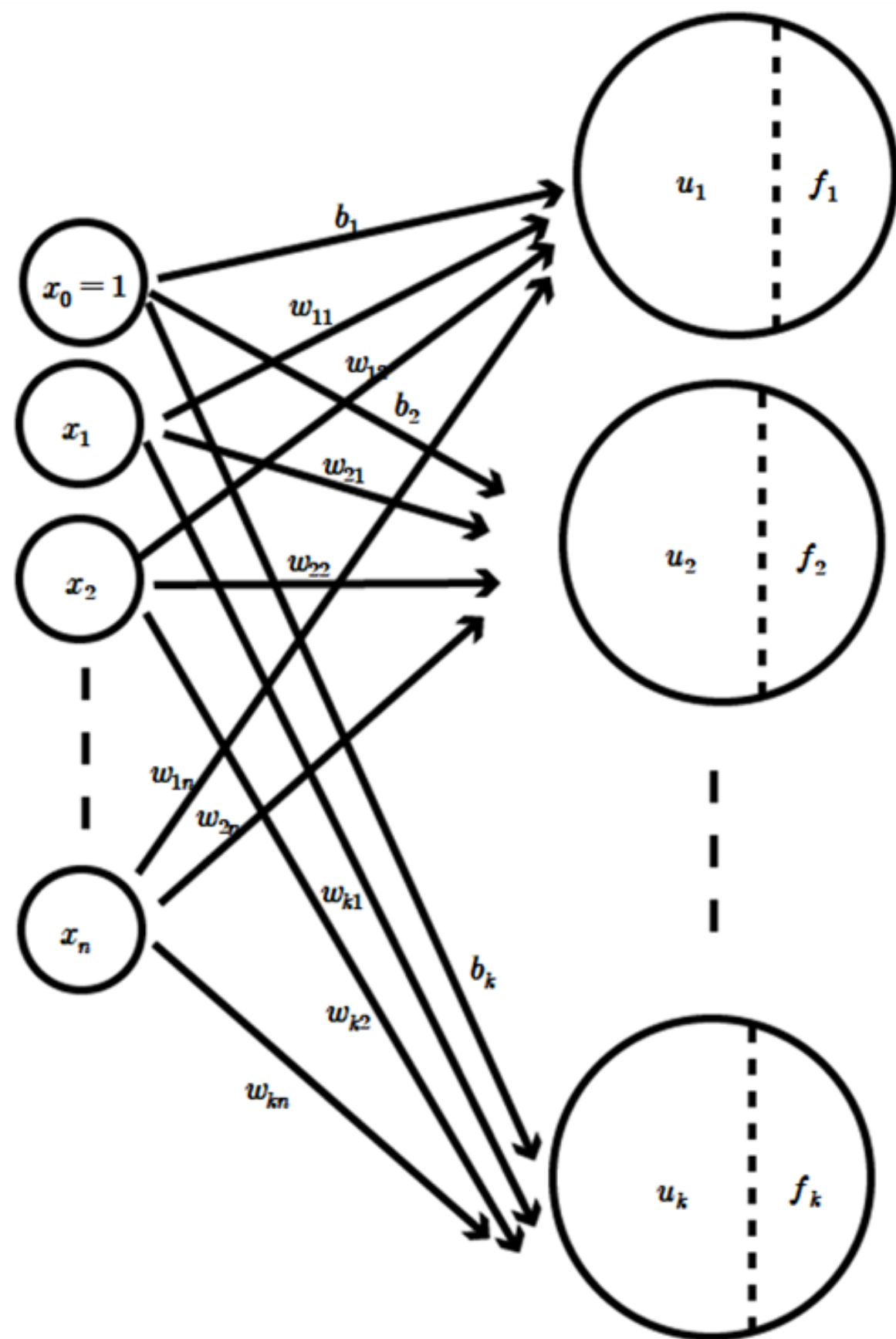
$$u_1 = w_{11}x_1 + w_{12}x_2 + \cdots + w_{1n}x_n + b_1$$

$$u_2 = w_{21}x_1 + w_{22}x_2 + \cdots + w_{2n}x_n + b_2$$

$\vdots$

$$u_k = w_{k1}x_1 + w_{k2}x_2 + \cdots + w_{kn}x_n + b_k$$

$$\begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_k \end{pmatrix} = \begin{pmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \cdots & \vdots \\ w_{k1} & w_{k2} & \cdots & w_{kn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_k \end{pmatrix} \Rightarrow \begin{pmatrix} f(u_1) \\ f(u_2) \\ \vdots \\ f(u_k) \end{pmatrix}$$





## 2. 활성화 함수

출력값이  $y = wx + b$ 의 형태로 나오는 경우도 있지만, 단순히 Yes, No 형태의 등의 다양한 출력값을 얻기 위하여  $y$  값을 다시 다른 함수를 통해 최종 출력값을 얻게 된다. 아래는  $y$  값을 가공하는 함수들인데 이를 보통 **활성화 함수(Active Function)**이라 부른다. 활성화 함수에는 아래 표와 같이  $Id$ ,  $ReLU$ ,  $\sigma$  등이 있습니다. 이들 중 하나를 표시할 때는 단순히  $g$ 로 표시하자.

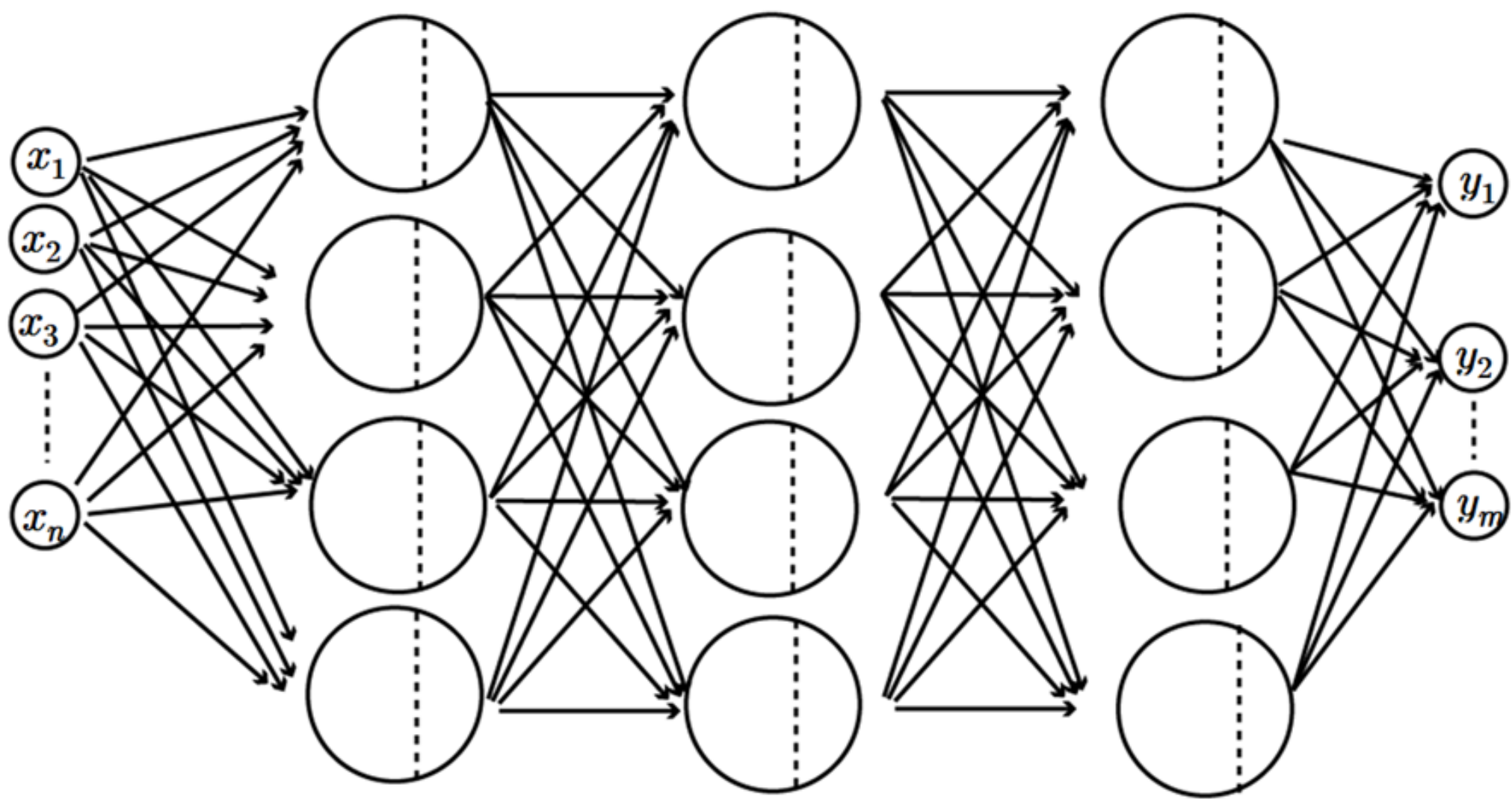
| Activation Functions |                                    |
|----------------------|------------------------------------|
| Function Name        | Formula                            |
| Identity             | $Id(y) = y$                        |
| ReLU                 | $ReLU(y) = \max(y, 0)$             |
| Sigmoid              | $\sigma(y) = \frac{1}{1 + e^{-y}}$ |

단순한 신경망의 단위구조의 수학적이 표시는  $g(f(x))$  형태의 표시이다.

즉,  $f(x)$ 와  $g(x)$ 의 합성함수 형태로 표시된다.



## 3. 다층 퍼셉트론 (Multi-Layer Perceptron, MLP)



입력층[Input Layer]

은닉층[Hidden Layer]

출력층[Output Layer]

수식 표현 (은닉층  $h$ , 출력층  $o$ ):

$$h_j = f\left(\sum_{i=1}^n w_{ij}x_i + b_j\right), \quad o_k = f\left(\sum_{j=1}^m w'_{jk}h_j + b'_k\right)$$

$w_{ij}$ : 입력층에서 은닉층으로의 가중치

$w'_{jk}$ : 은닉층에서 출력층으로의 가중치

$b_j, b'_k$ : 절편(편향)

$h_j$ : 은닉층의 뉴런 출력

$o_k$ : 출력층의 뉴런 출력



## 4. 비용 함수 (Cost Function)

비용 함수는 신경망의 예측값과 실제값의 차이를 측정하여, 신경망의 성능을 평가하는 데 사용된다. 대표적인 비용 함수는 평균 제곱 오차(Mean Squared Error, MSE)와 교차 엔트로피(Cross-Entropy)이다.

평균 제곱 오차 (MSE):

$$E = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

$y_i$ : 실제값

$\hat{y}_i$ : 예측값

교차 엔트로피 (Cross-Entropy):

$$E = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$



## 5. 역전파 알고리즘 (Backpropagation)

역전파 알고리즘은 신경망의 가중치를 업데이트하여 오차를 최소화하는 방법이다. 이 알고리즘은 오차를 출력층에서 입력층으로 역방향으로 전파시키며, 각 층의 가중치와 편향을 조정한다.

단계:

1. 순전파 (Feedforward): 입력 데이터를 통해 신경망을 순차적으로 통과시키며 출력을 계산한다.
2. 오차 계산 (Error Calculation): 출력층에서 오차를 계산한다.

$$E = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

3. 역전파 (Backpropagation): 실제 출력이 목표 출력에 더 가까워지도록 네트워크의 각 가중치를 업데이트하여 각 출력 뉴런과 네트워크 전체의 오류를 최소화하는 것이다.
4. 가중치 업데이트 (Weight Update): 경사 하강법(Gradient Descent)을 사용하여 가중치와 편향을 업데이트한다.

$$w_{ij}^+ = w_{ij} - \alpha \frac{\partial E}{\partial w_{ij}}, \quad b_j^+ = b_j - \alpha \frac{\partial E}{\partial b_j}$$

### 3. 역전파 알고리즘

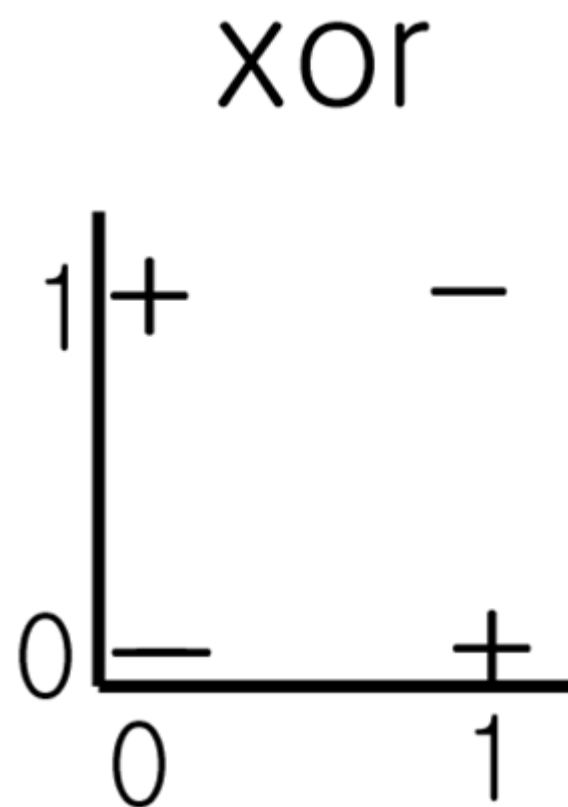


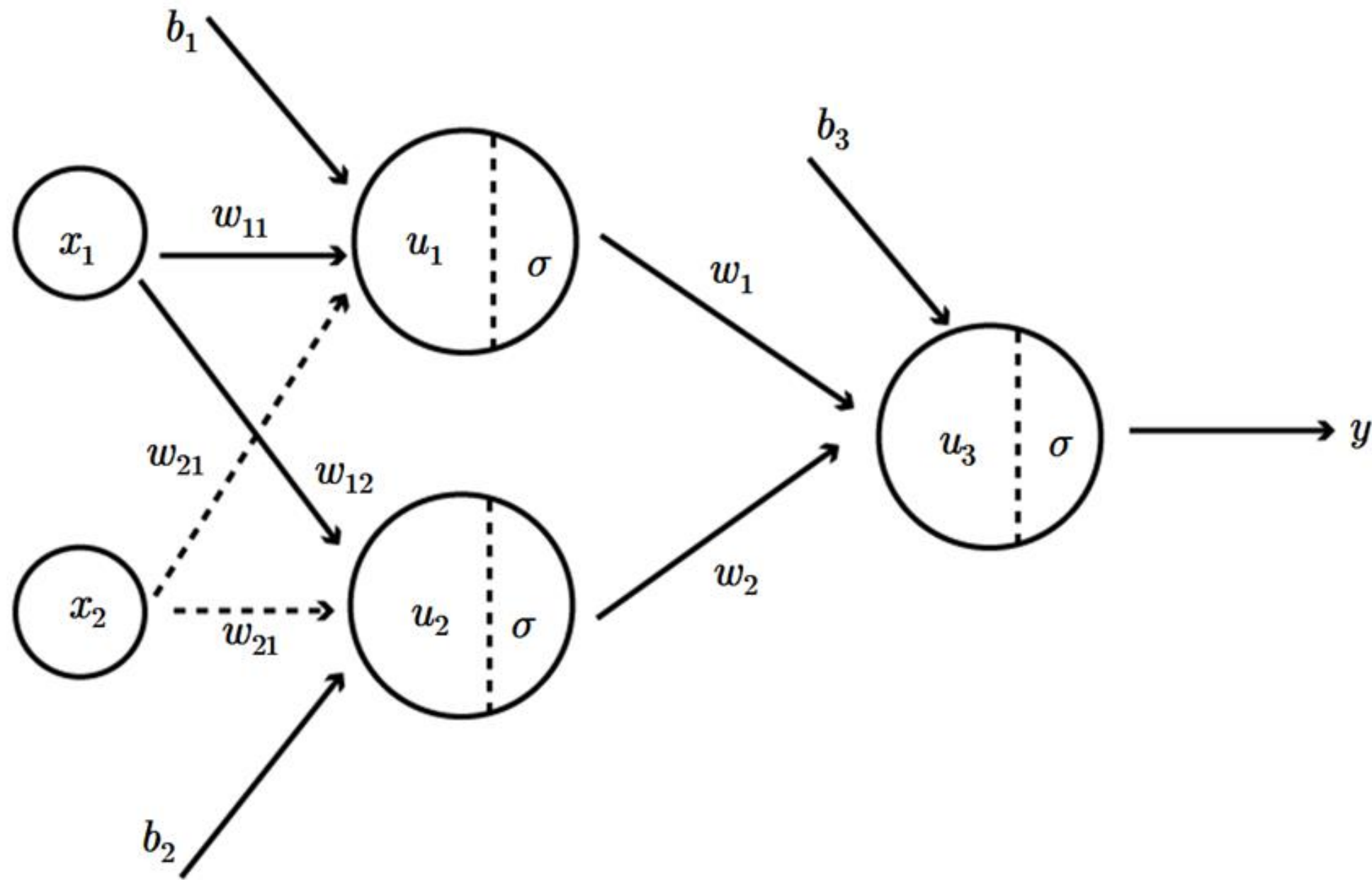
## 1. 순전파(Forward Propagation)

순전파(Forward Propagation)는 인공 신경망에서 입력 데이터를 받아 출력 값을 계산하는 과정이다. 이 과정은 입력층에서 시작하여 출력층까지 각 층을 거치면서 데이터를 처리한다. 순전파의 주요 목표는 주어진 입력 데이터에 대해 신경망의 예측 값을 얻는 것이다.

앞에서 언급한 *XOR*문제를 가지고 순전파 과정을 설명해보자.

| $x_1$ | $x_2$ | <i>XOR</i> |   |
|-------|-------|------------|---|
| 0     | 0     | 0          | - |
| 0     | 1     | 1          | + |
| 1     | 0     | 1          | + |
| 1     | 1     | 0          | - |



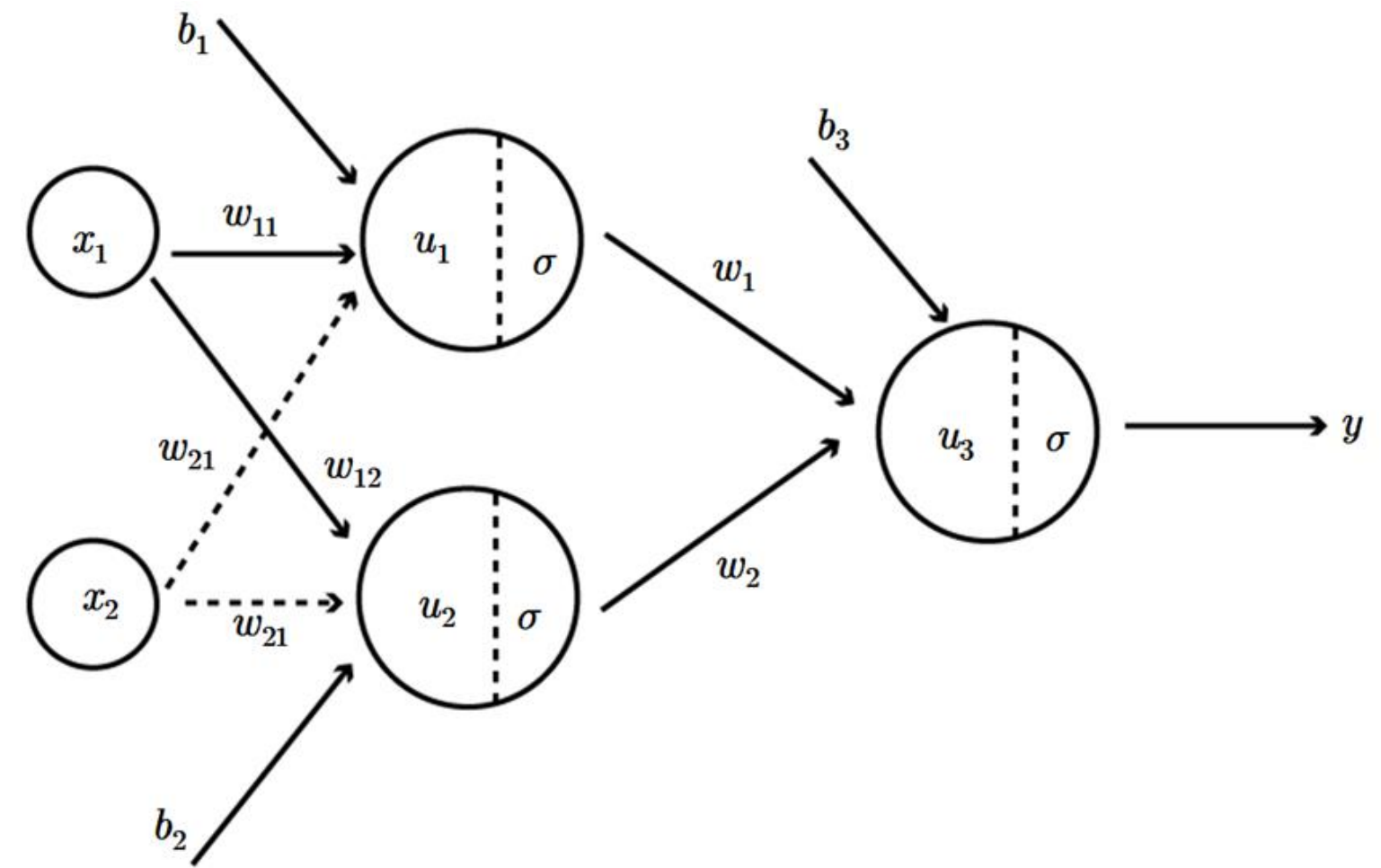
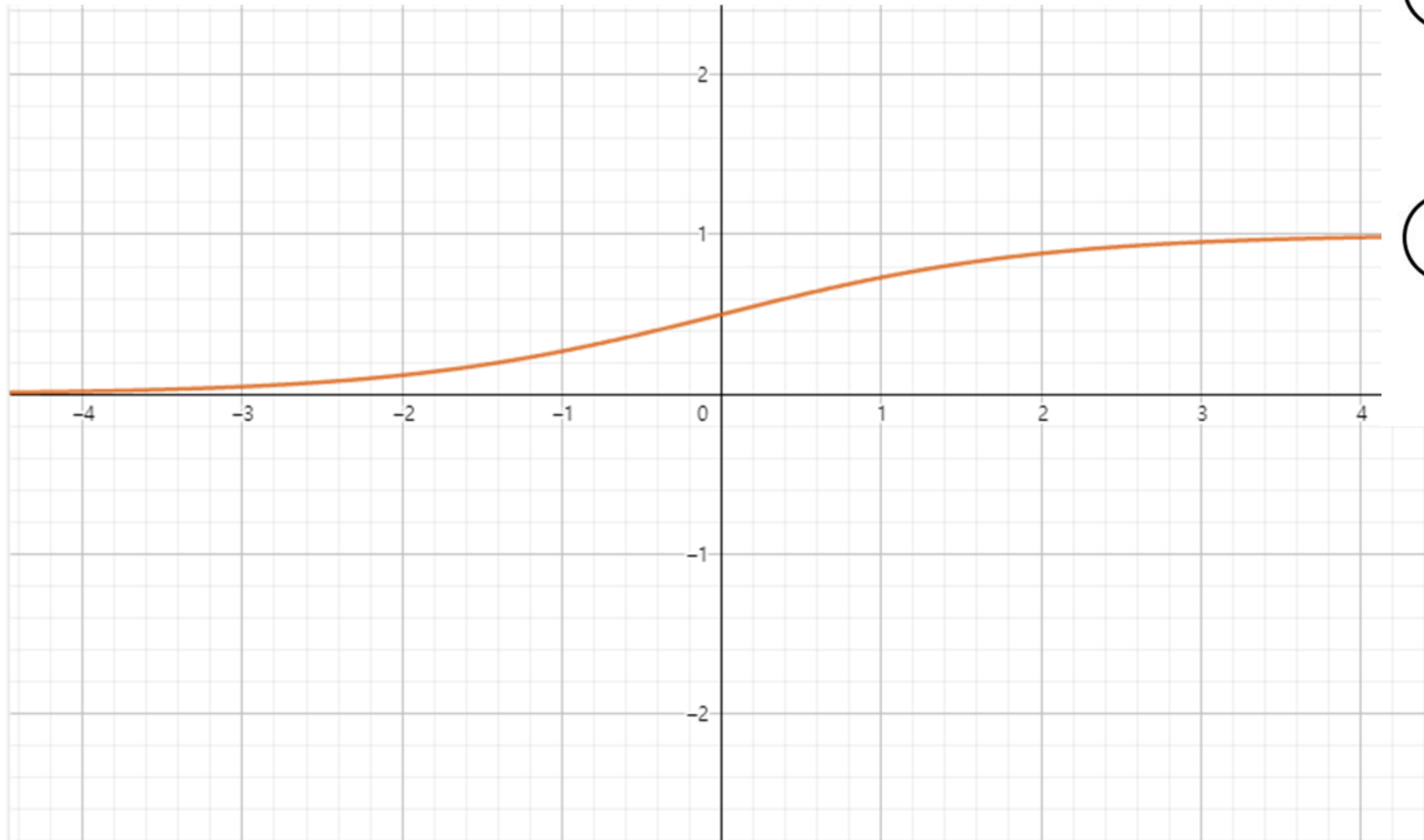


입력:  $x_1, x_2$

$$\begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = \begin{pmatrix} u_1 \\ u_2 \end{pmatrix}$$



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



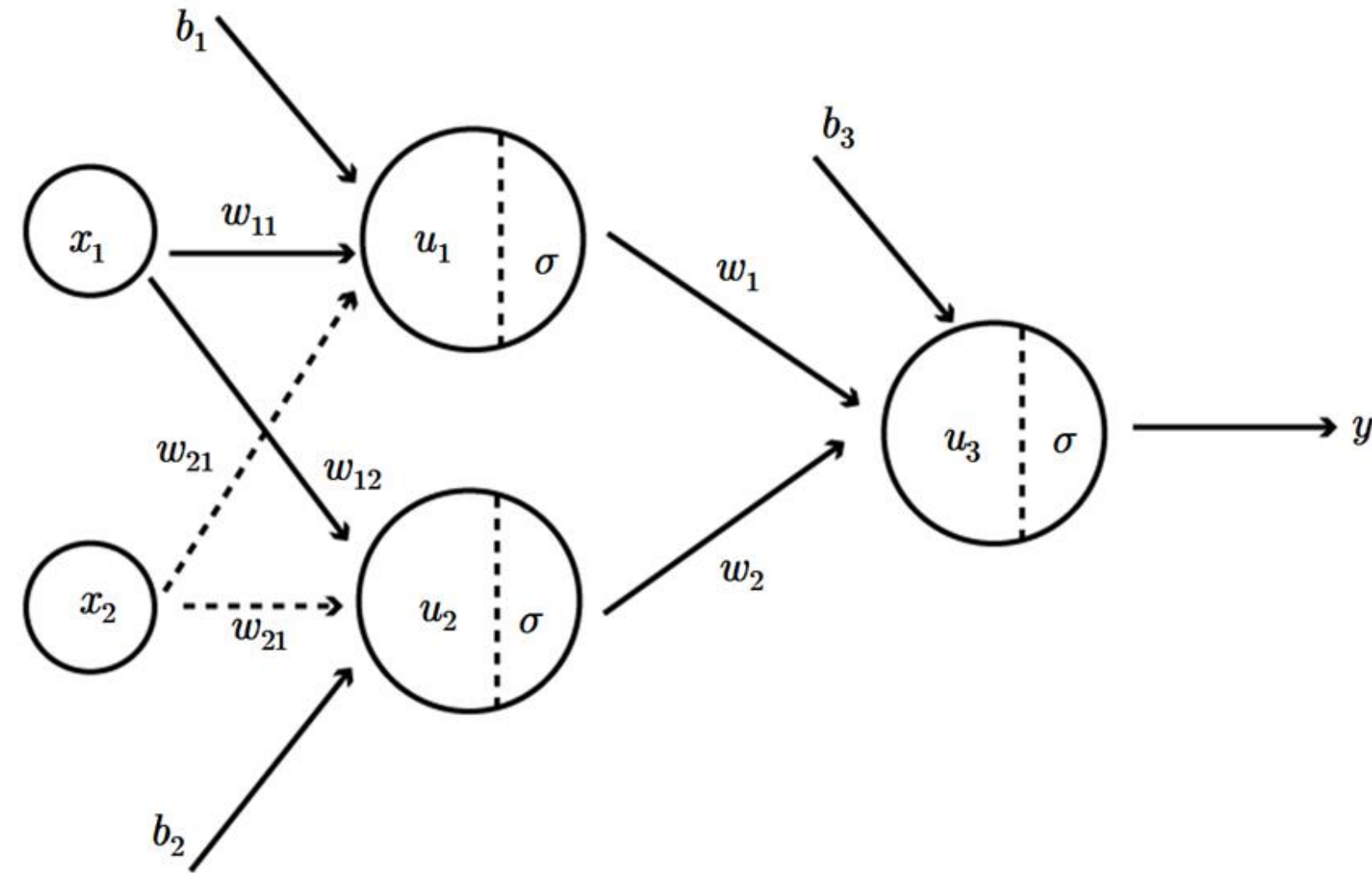


$$\sigma(u_1), \sigma(u_2)$$

$$(w_1 \ w_2) \begin{pmatrix} \sigma(u_1) \\ \sigma(u_2) \end{pmatrix} + b_3 = w_1 \sigma(u_1) + w_2 \sigma(u_2)$$

$$\begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{pmatrix} = \begin{pmatrix} 20 & 20 \\ -20 & -20 \end{pmatrix}, \quad \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = \begin{pmatrix} -10 \\ 30 \end{pmatrix}$$

$$(w_1 \ w_2) = (20 \ 20), \quad b_3 = -30$$





$$(1) \begin{pmatrix} 0 \\ 0 \end{pmatrix}: \begin{pmatrix} 20 & 20 \\ -20 & -20 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \begin{pmatrix} -10 \\ 30 \end{pmatrix} = \begin{pmatrix} -10 \\ 30 \end{pmatrix}$$

$$\begin{pmatrix} \sigma(-10) \\ \sigma(30) \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$(20 \ 20) \begin{pmatrix} 0 \\ 1 \end{pmatrix} - 30 = -10 \Rightarrow \sigma(-10) = 0$$

$$(2) \begin{pmatrix} 1 \\ 0 \end{pmatrix}: \begin{pmatrix} 20 & 20 \\ -20 & -20 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} -10 \\ 30 \end{pmatrix} = \begin{pmatrix} 20 \\ -20 \end{pmatrix} + \begin{pmatrix} -10 \\ 30 \end{pmatrix} = \begin{pmatrix} 10 \\ 10 \end{pmatrix}$$

$$\begin{pmatrix} \sigma(10) \\ \sigma(10) \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

$$(20 \ 20) \begin{pmatrix} 1 \\ 1 \end{pmatrix} - 30 = 10 \Rightarrow \sigma(10) = 1$$

| $x_1$ | $x_2$ | $XOR$ |   |
|-------|-------|-------|---|
| 0     | 0     | 0     | - |
| 0     | 1     | 1     | + |
| 1     | 0     | 1     | + |
| 1     | 1     | 0     | - |

$$\begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{pmatrix} = \begin{pmatrix} 20 & 20 \\ -20 & -20 \end{pmatrix}, \quad \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = \begin{pmatrix} -10 \\ 30 \end{pmatrix}$$

$$(w_1 \ w_2) = (20 \ 20), \quad b_3 = -30$$



$$(3) \begin{pmatrix} 0 \\ 1 \end{pmatrix} : \begin{pmatrix} 20 & 20 \\ -20 & -20 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} + \begin{pmatrix} -10 \\ 30 \end{pmatrix} = \begin{pmatrix} 20 \\ -20 \end{pmatrix} + \begin{pmatrix} -10 \\ 30 \end{pmatrix} = \begin{pmatrix} 10 \\ 10 \end{pmatrix}$$

$$\begin{pmatrix} \sigma(10) \\ \sigma(10) \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

$$(20 \ 20) \begin{pmatrix} 1 \\ 1 \end{pmatrix} - 30 = 10 \Rightarrow \sigma(10) = 1$$

$$(4) \begin{pmatrix} 1 \\ 1 \end{pmatrix} : \begin{pmatrix} 20 & 20 \\ -20 & -20 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} + \begin{pmatrix} -10 \\ 30 \end{pmatrix} = \begin{pmatrix} 40 \\ -40 \end{pmatrix} + \begin{pmatrix} -10 \\ 30 \end{pmatrix} = \begin{pmatrix} 30 \\ -10 \end{pmatrix}$$

$$\begin{pmatrix} \sigma(30) \\ \sigma(-10) \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$(20 \ 20) \begin{pmatrix} 1 \\ 0 \end{pmatrix} - 30 = -10 \Rightarrow \sigma(-10) = 0$$

| $x_1$ | $x_2$ | $XOR$ |   |
|-------|-------|-------|---|
| 0     | 0     | 0     | - |
| 0     | 1     | 1     | + |
| 1     | 0     | 1     | + |
| 1     | 1     | 0     | - |

$$\begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{pmatrix} = \begin{pmatrix} 20 & 20 \\ -20 & -20 \end{pmatrix}, \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = \begin{pmatrix} -10 \\ 30 \end{pmatrix}$$

$$(w_1 \ w_2) = (20 \ 20), \ b_3 = -30$$



## 2.역전파 알고리즘

우리가 어떤 목적을 수행하기 위해서 하나의 신경망을 구성했다고 생각하자. 하나의 신경망을 구성하면 신경망 안에 있는 가중치 값들과 절편값들 적절히 훈련을 해야 한다. 훈련을 하기 위해서는 훈련에 맞는 비용함수를 정의한다. 그러면 정의된 비용함수는 가중치값들과 절편값들을 변수로 가진다. 가중치 값들과 절편값의 초깃값을 정하고, 이 값으로부터 시작하여 점진적으로 비용함수값이 적어지도록 가중치값들과 절편값을 Gradient Descent 탐색방법을 사용하여 찾아나간다. 이때 chain rule을 사용하여 뒤에서 부터 앞으로 변수값을 계산하기 때문에 이를 흔히 역전파 알고리즘(backpropagation algorithm)이라고 부른다.

역전파 알고리즘을 구체적으로 설명하기 위해서 아주 간단한 예를 가지고 설명하려고 한다.



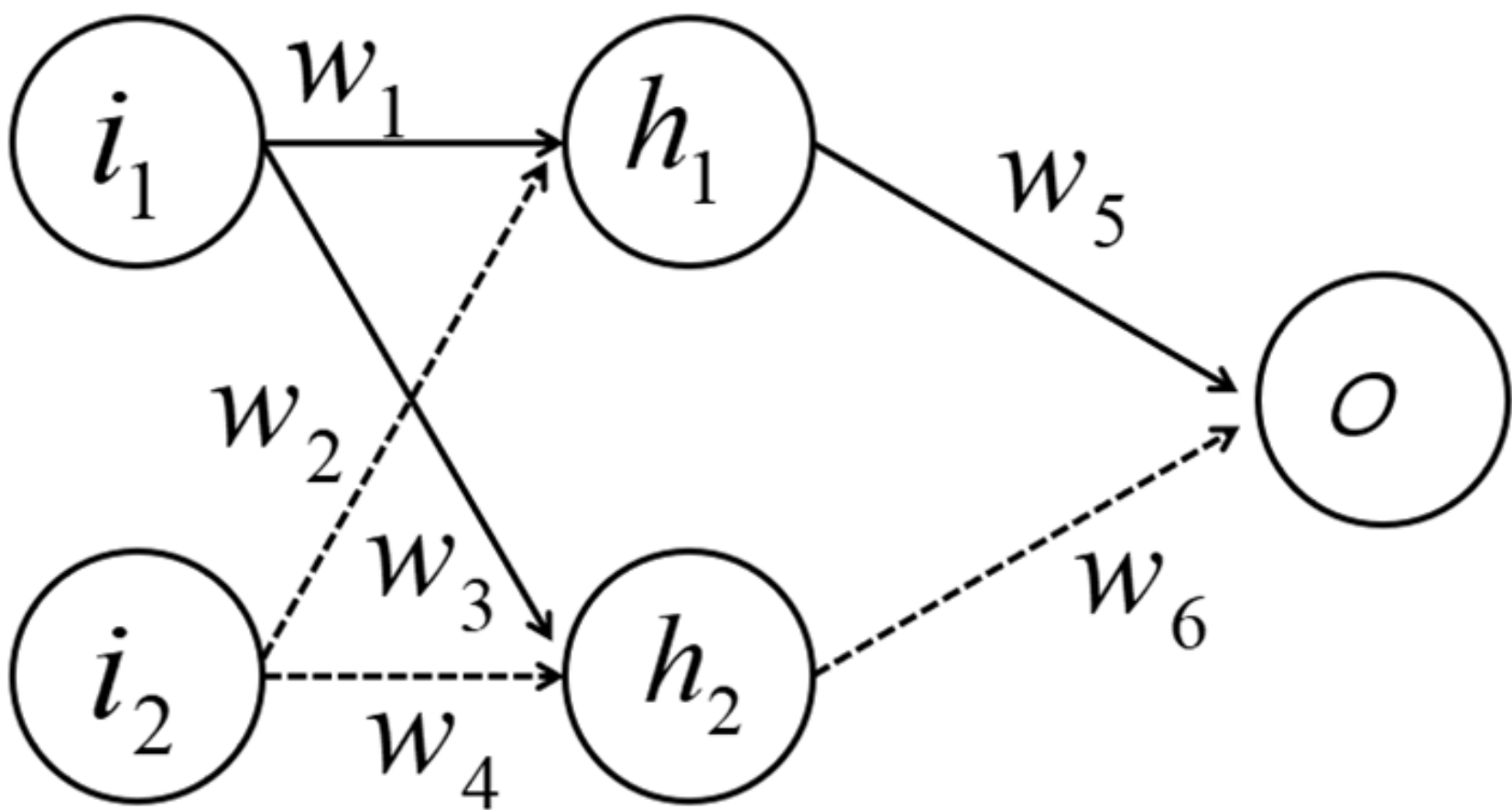
## 예제1: 역전파 알고리즘

머신러닝을 위한  
선형대수학

12 신경망의 기본 구조와  
행렬 연산

3. 역전파 알고리즘

**예제1:** 이 예제는 입력 뉴런이 2개, 은닉뉴런이 2개 출력뉴런이 1개이다.



$$h_1 = i_1 \cdot w_1 + i_2 \cdot w_2, \quad h_2 = i_1 \cdot w_3 + i_2 \cdot w_4$$

$$o = h_1 \cdot w_5 + h_2 \cdot w_6$$

목표는  $i_1 = 2$ ,  $i_2 = 3$ 일 때  $result = 1$ 이 나오도록

$w_1, w_2, w_3, w_4, w_5, w_6$ 의 값을 결정하는 것이다.

이를 위해 비용함수  $E$ 는 다음과 같이 정의된다.

$$\text{비용함수: } E = \frac{1}{2}(o - result)^2 = \frac{1}{2}(o - 1)^2$$



## 예제1: 역전파 알고리즘

머신러닝을 위한  
선형대수학

12 신경망의 기본 구조와  
행렬 연산

3. 역전파 알고리즘

$i_1 = 2, i_2 = 3$ 을 대입해 보면

$h_1 = 2w_1 + 3w_2, h_2 = 2w_3 + 3w_4$ 이고

$o = h_1 \cdot w_5 + h_2 \cdot w_6$

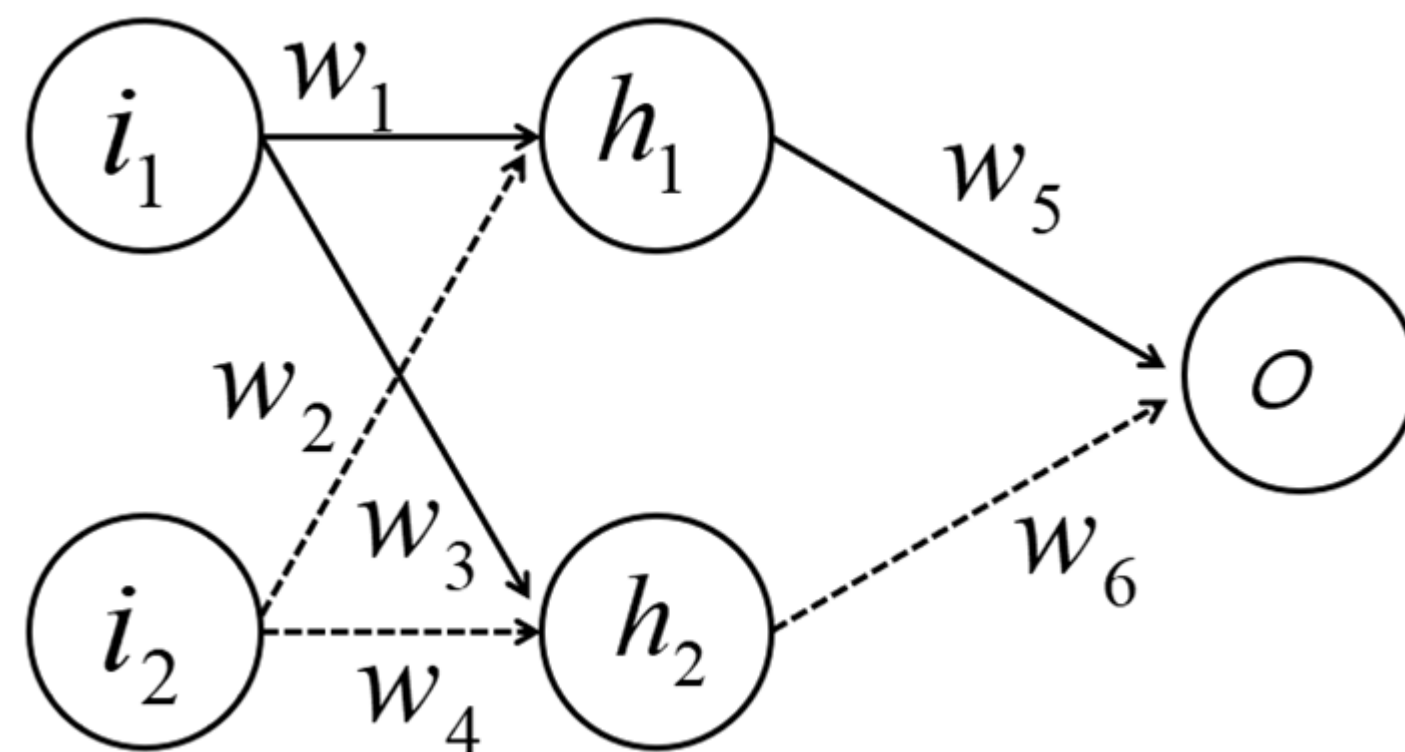
$= (2w_1 + 3w_2)w_5 + (2w_3 + 3w_4)w_6$

$= 2w_1w_5 + 3w_2w_5 + 2w_3w_6 + 3w_4w_6$

따라서

$$E = \frac{1}{2}(o - result)^2 = \frac{1}{2}(o - 1)^2$$

$$= \frac{1}{2}(2w_1w_5 + 3w_2w_5 + 2w_3w_6 + 3w_4w_6 - 1)^2$$





먼저 편미분을 이용한 Gradient descent 방법을 이용하여  $w_i, i = 1, 2, \dots, 6$  값들을 조정해 보자.

$E : R^6 \rightarrow R : E(w_1, w_2, w_3, w_4, w_5, w_6)$ 인 함수이다.

$w = (w_1, w_2, w_3, w_4, w_5, w_6)$ 라 하자.

우리의 목적은  $w_0 = (w_{01}, w_{02}, w_{03}, w_{04}, w_{05}, w_{06})$ 으로 시작하여

$$E(w_0) > E(w_1) > E(w_2) > \dots$$

가 되도록  $w_1, w_2, \dots$ 을 구하는 것이다.

$w_1 = w_0 - \alpha \nabla E(w_0)$ 인데 여기서  $0 < \alpha < 1$ 라 하자.

$$\nabla E(w_0) = \left( \frac{\partial E(w_0)}{\partial w_1}, \frac{\partial E(w_0)}{\partial w_2}, \dots, \frac{\partial E(w_0)}{\partial w_6} \right)$$



## 예제1: 역전파 알고리즘

머신러닝을 위한  
선형대수학

12 신경망의 기본 구조와  
행렬 연산

3. 역전파 알고리즘

$$\frac{\partial E}{\partial w_1} = (o - 1) \frac{\partial o}{\partial w_1} = (o - 1) \cdot 2w_5$$

$$\frac{\partial E}{\partial w_2} = (o - 1) \frac{\partial o}{\partial w_2} = (o - 1) \cdot 3w_5$$

$$\frac{\partial E}{\partial w_3} = (o - 1) \frac{\partial o}{\partial w_3} = (o - 1) \cdot 2w_6$$

$$\frac{\partial E}{\partial w_4} = (o - 1) \frac{\partial o}{\partial w_4} = (o - 1) \cdot 3w_6$$

$$\frac{\partial E}{\partial w_5} = (o - 1) \frac{\partial o}{\partial w_5} = (o - 1) \cdot (2w_1 + 3w_2)$$

$$\frac{\partial E}{\partial w_6} = (o - 1) \frac{\partial o}{\partial w_6} = (o - 1) \cdot (2w_3 + 3w_4)$$

$$\begin{aligned} E &= \frac{1}{2} (o - result)^2 = \frac{1}{2} (o - 1)^2 \\ &= \frac{1}{2} (2w_1w_5 + 3w_2w_5 + 2w_3w_6 + 3w_4w_6 - 1)^2 \end{aligned}$$



$$\begin{aligned}w_1 &= w_0 - \alpha \nabla E(w_0) \\&= (w_{01}, w_{02}, w_{03}, w_{04}, w_{05}, w_{06}) - \alpha \left( \frac{\partial E(w_0)}{\partial w_1}, \frac{\partial E(w_0)}{\partial w_2}, \dots, \frac{\partial E(w_0)}{\partial w_6} \right) \\&= \left( w_{01} - \alpha \frac{\partial E(w_0)}{\partial w_1}, w_{02} - \alpha \frac{\partial E(w_0)}{\partial w_2}, \dots, w_{06} - \alpha \frac{\partial E(w_0)}{\partial w_6} \right)\end{aligned}$$

$w_1 = (w_{11}, w_{12}, w_{13}, w_{14}, w_{15}, w_{16})$ 라 하면

$$\begin{aligned}w_{11} &= w_{01} - \alpha \frac{\partial E(w_0)}{\partial w_1}, \quad w_{12} = w_{02} - \alpha \frac{\partial E(w_0)}{\partial w_2}, \quad w_{13} = w_{03} - \alpha \frac{\partial E(w_0)}{\partial w_3}, \\w_{14} &= w_{04} - \alpha \frac{\partial E(w_0)}{\partial w_4}, \quad w_{15} = w_{05} - \alpha \frac{\partial E(w_0)}{\partial w_5}, \quad w_{16} = w_{06} - \alpha \frac{\partial E(w_0)}{\partial w_6}\end{aligned}$$

으로 업데이트 된다.



초깃값을

$$w_{01} = 0.11, w_{02} = 0.21, w_{03} = 0.12, w_{04} = 0.08, w_{05} = 0.14, w_{06} = 0.15$$

라 하자. 이때  $E = \frac{1}{2}(o - result)^2 = \frac{1}{2}(o - 1)^2 = 0.32724$ 이다.

100번 반복 후  $E = 3.3455 \times 10^{-6} = 0.0000033455$ 이다.

이때,  $w_1 = 0.2241, w_2 = 0.3812, w_3 = 0.2198,$

$w_4 = 0.2297, w_5 = 0.3994, w_6 = 0.3205$ 이다.



## 2. Backpropagation 방법

Step 1:  $w_5, w_6$  업데이트

$$i_1 = 2, i_2 = 3, result = 1$$

초깃값을

$$w_{01} = 0.11, w_{02} = 0.21, w_{03} = 0.12, w_{04} = 0.08, w_{05} = 0.14, w_{06} = 0.15$$

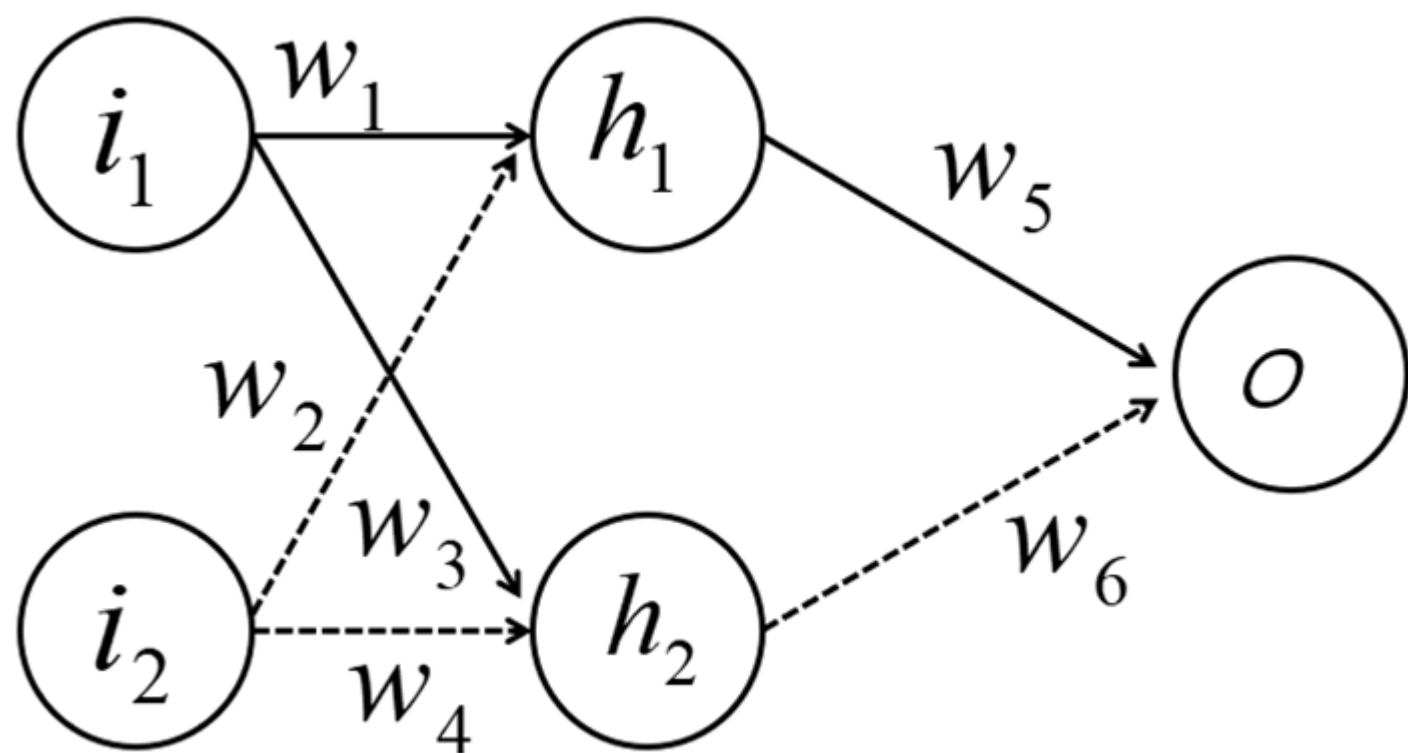
라 하자.

$$h_1 = 2w_1 + 3w_2 = 2 \times 0.11 + 3 \times 0.21 = 0.85,$$

$$h_2 = 2w_3 + 3w_4 = 2 \times 0.12 + 3 \times 0.08 = 0.48$$

$$o = h_1 \cdot w_5 + h_2 \cdot w_6 = 0.85 \times 0.14 + 0.48 \times 0.15 = 0.191$$

$$E = \frac{1}{2}(o - result)^2 = \frac{1}{2}(o - 1)^2 = 0.32724 \text{이다.}$$





# 예제1: 역전파 알고리즘

머신러닝을 위한  
선형대수학

12 신경망의 기본 구조와  
행렬 연산

3. 역전파 알고리즘

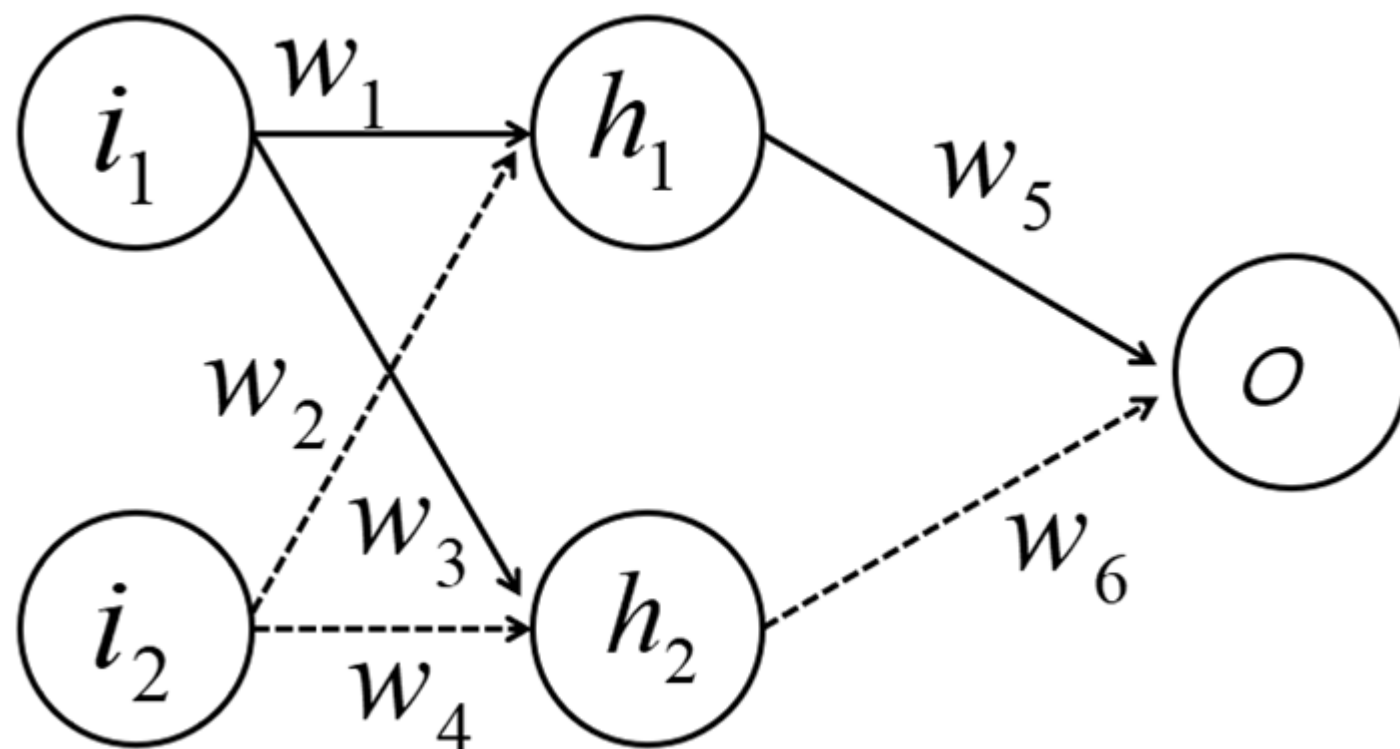
$$E = \frac{1}{2}(o - 1)^2, \quad o = h_1 \cdot w_5 + h_2 \cdot w_6$$

$$\frac{\partial E}{\partial w_5} = \frac{\partial E}{\partial o} \frac{\partial o}{\partial w_5} = (o - 1)h_1 = (0.191 - 1) \times 0.85 = -0.68765$$

$$w_5^+ = w_5 - \alpha \frac{\partial E}{\partial w_5} = w_5 - \alpha^* (o - 1)h_1 = 0.14 - \alpha^* (-0.68765)$$

$$\frac{\partial E}{\partial w_6} = \frac{\partial E}{\partial o} \frac{\partial o}{\partial w_6} = (o - 1)h_2 = (0.191 - 1) \times 0.48 = -0.38832$$

$$w_6^+ = w_6 - \alpha \frac{\partial E}{\partial w_6} = w_6 - \alpha^* (o - 1)h_2 = 0.15 - \alpha^* (-0.38832)$$





# 예제1: 역전파 알고리즘

머신러닝을 위한  
선형대수학

12 신경망의 기본 구조와  
행렬 연산

3. 역전파 알고리즘

Step 2:  $w_1, w_2, w_3, w_4$  업데이트

$$E = \frac{1}{2}(o - 1)^2, \quad o = h_1 \cdot w_5 + h_2 \cdot w_6$$

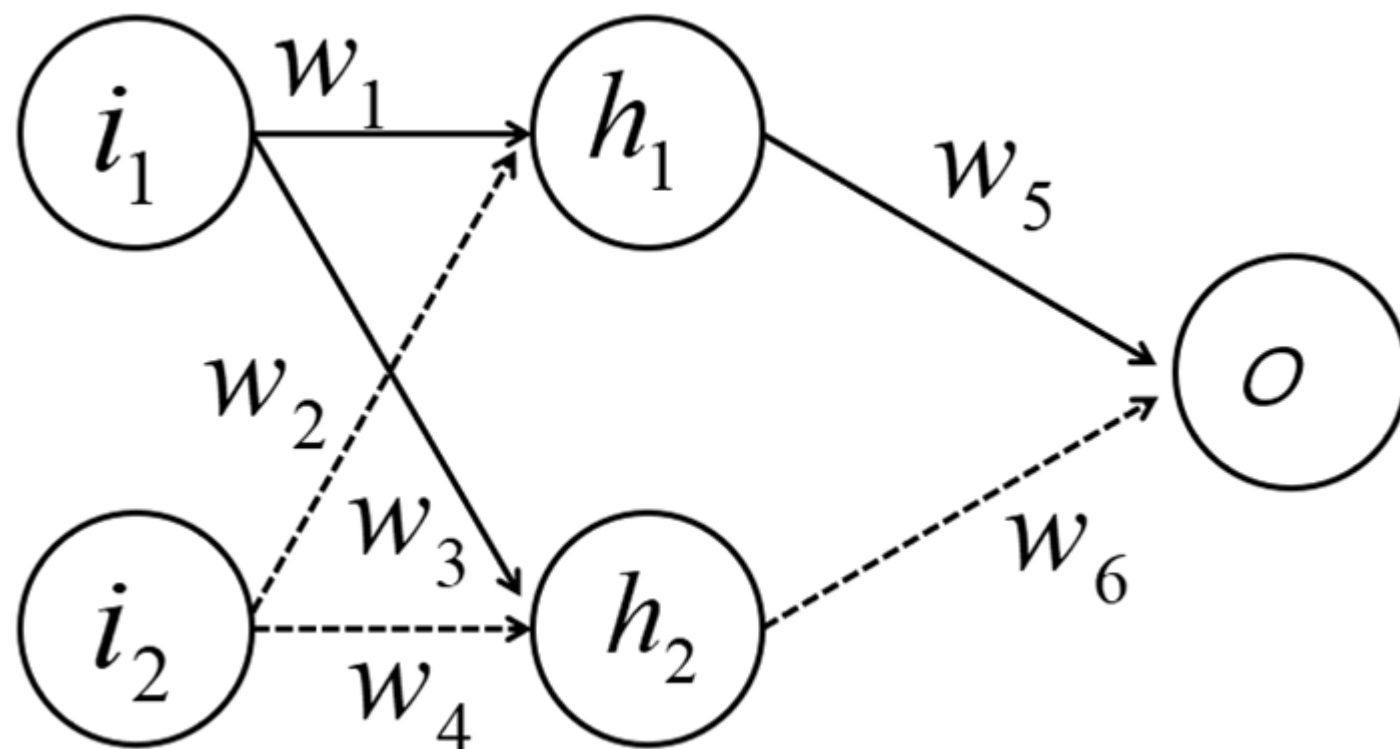
$$h_1 = 2w_1 + 3w_2, \quad h_2 = 2w_3 + 3w_4$$

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial o} \cdot \frac{\partial o}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1} = (o - 1) \cdot w_5 \cdot 2,$$

$$\frac{\partial E}{\partial w_2} = \frac{\partial E}{\partial o} \cdot \frac{\partial o}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_2} = (o - 1) \cdot w_5 \cdot 3,$$

$$\frac{\partial E}{\partial w_3} = \frac{\partial E}{\partial o} \cdot \frac{\partial o}{\partial h_2} \cdot \frac{\partial h_2}{\partial w_3} = (o - 1) \cdot w_6 \cdot 2,$$

$$\frac{\partial E}{\partial w_4} = \frac{\partial E}{\partial o} \cdot \frac{\partial o}{\partial h_2} \cdot \frac{\partial h_2}{\partial w_4} = (o - 1) \cdot w_6 \cdot 3$$





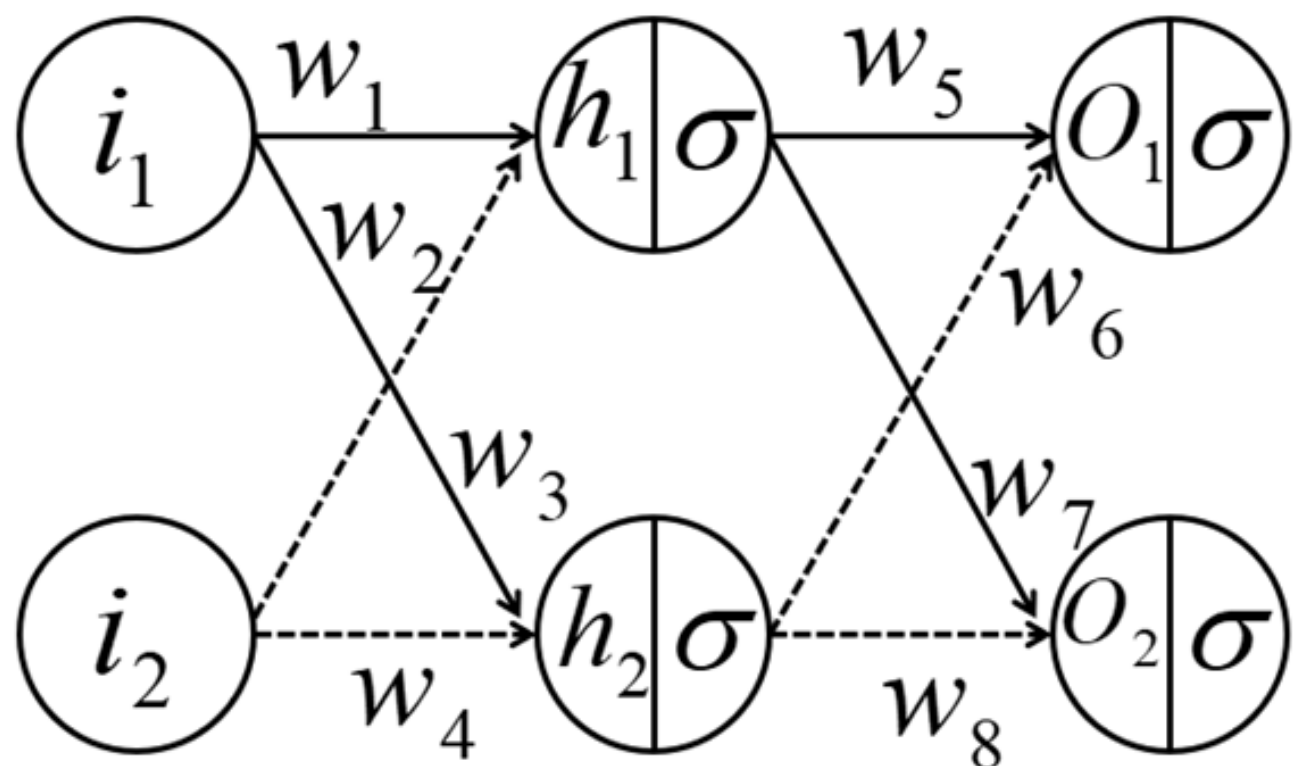
## 예제2: 역전파 알고리즘

머신러닝을 위한  
선형대수학

12 신경망의 기본 구조와  
행렬 연산

3. 역전파 알고리즘

**예제2:** 이번 예제는 입력 뉴런이 2개, 은닉뉴런이 2개 출력뉴런이 2개이고 이번에도 단순 계산을 위하여 편의는 생략하였다.



$$h_1 = w_1 \cdot i_1 + w_2 \cdot i_2, \quad h_2 = w_3 \cdot i_1 + w_4 \cdot i_2$$

$$outh_1 = \sigma(h_1) = \frac{1}{1 + e^{-h_1}}, \quad outh_2 = \sigma(h_2) = \frac{1}{1 + e^{-h_2}}$$

$$o_1 = w_5 \cdot outh_1 + w_6 \cdot outh_2, \quad o_2 = w_7 \cdot outh_1 + w_8 \cdot outh_2$$

$$outo_1 = \sigma(o_1) = \frac{1}{1 + e^{-o_1}}, \quad outo_2 = \sigma(o_2) = \frac{1}{1 + e^{-o_2}}$$



조건:  $i_1 = 0.05$ ,  $i_2 = 0.1$  일 때

결과:  $result_1 = 0.01$ ,  $result_2 = 0.99$

을 낼 수 있도록  $w_1, w_2, \dots, w_8$ 의 최적화된 값을 찾아낸다.

**비용함수:**  $E = E_1 + E_2 = \frac{1}{2}(outo_1 - result_1)^2 + \frac{1}{2}(outo_2 - result_2)^2$

이때, 비용함수를 좀 자세히 보면  $E: R^8 \rightarrow R$ :

$$E(w_1, w_2, \dots, w_8)$$

$$= E_1(w_1, w_2, w_3, w_4, w_5, w_6, w_7, w_8) + E_2(w_1, w_2, w_3, w_4, w_5, w_6, w_7, w_8)$$

$$= \frac{1}{2}(outo_1 - result_1)^2 + \frac{1}{2}(outo_2 - result_2)^2$$

$$= \frac{1}{2}(outo_1 - 0.01)^2 + \frac{1}{2}(outo_2 - 0.99)^2$$



## 예제2: 역전파 알고리즘

머신러닝을 위한  
선형대수학

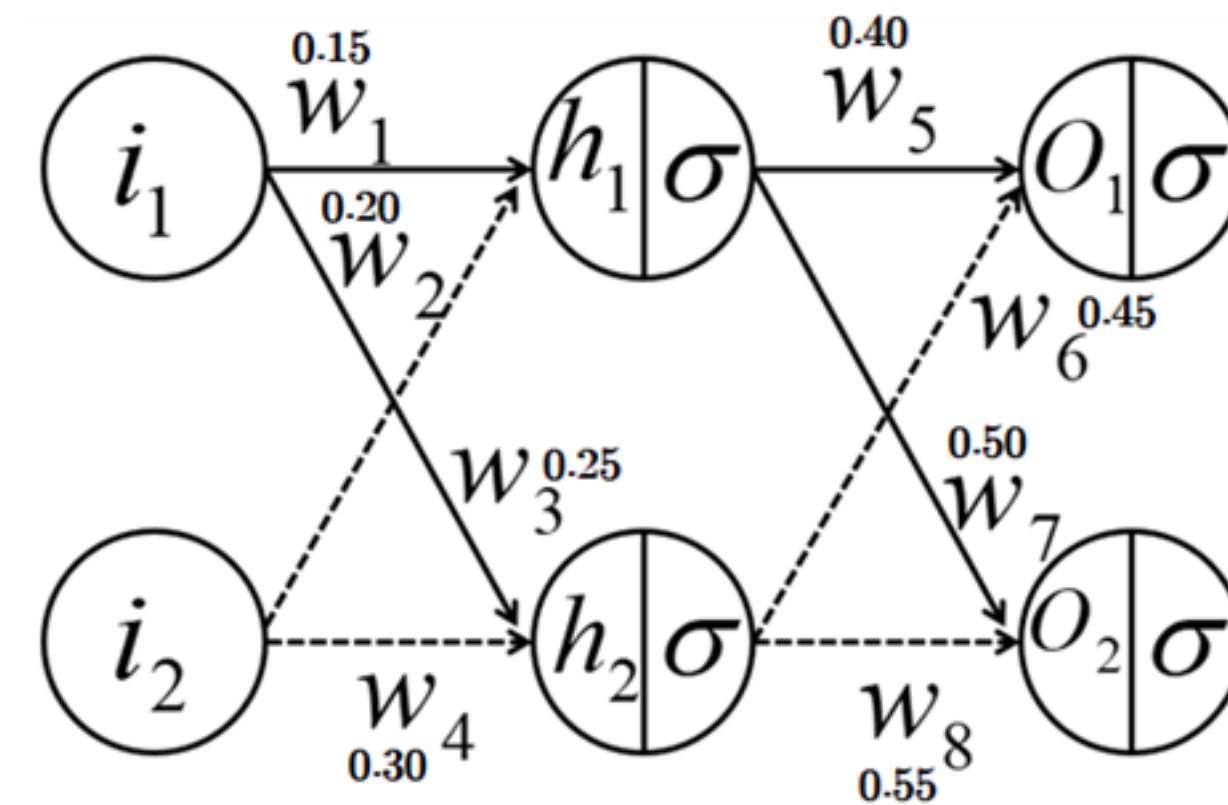
12 신경망의 기본 구조와  
행렬 연산

3. 역전파 알고리즘

**Step 1: 초기조건:** 아래와 같이 8개의 초깃값이 주어졌다.

$$w_{01} = 0.15, w_{02} = 0.20, w_{03} = 0.25, w_{04} = 0.30,$$

$$w_{05} = 0.40, w_{06} = 0.45, w_{07} = 0.50, w_{08} = 0.55$$





Step 2:  $outo_1$ 과  $outo_2$ 값을 계산해 보자.

$h_1 = w_1 \cdot i_1 + w_2 \cdot i_2$ 에서

$$h_1 = 0.15 * 0.05 + 0.2 * 0.1 = \mathbf{0.0275}$$

$$outh_1 = \sigma(h_1) = \frac{1}{1 + e^{-h_1}} = \frac{1}{1 + e^{-0.0275}} = \mathbf{0.5069}$$

$h_2 = w_3 \cdot i_1 + w_4 \cdot i_2$ 에서

$$h_2 = 0.25 * 0.05 + 0.30 * 0.1 = \mathbf{0.0425}$$

$$outh_2 = \sigma(h_2) = \frac{1}{1 + e^{-h_2}} = \frac{1}{1 + e^{-0.0425}} = \mathbf{0.5106}$$



## 예제2: 역전파 알고리즘

머신러닝을 위한  
선형대수학

12 신경망의 기본 구조와  
행렬 연산

3. 역전파 알고리즘

$$o_1 = w_5 \cdot outh_1 + w_6 \cdot outh_2$$

$$o_1 = 0.4 \cdot 0.5069 + 0.45 \cdot 0.5106 = \mathbf{0.4325}$$

$$outo_1 = \sigma(o_1) = \frac{1}{1 + e^{-o_1}} = \frac{1}{1 + e^{-0.4325}} = \mathbf{0.6065}$$

$$o_2 = w_7 \cdot outh_1 + w_8 \cdot outh_2$$

$$o_2 = 0.50 \cdot 0.5069 + 0.55 \cdot 0.5106 = \mathbf{0.5343}$$

$$outo_2 = \sigma(o_2) = \frac{1}{1 + e^{-o_2}} = \frac{1}{1 + e^{-0.5343}} = \mathbf{0.6305}$$

$$\begin{aligned} E(w_{01}, w_{02}, \dots, w_{08}) &= \frac{1}{2} (outo_1 - 0.01)^2 + \frac{1}{2} (outo_2 - 0.99)^2 \\ &= \frac{1}{2} (0.6065 - 0.01)^2 + \frac{1}{2} (0.6305 - 0.99)^2 \\ &= 0.1779 + 0.1461 = \mathbf{0.3240} \end{aligned}$$



다음 순서는  $E(w_{01}, w_{02}, \dots, w_{08}) > E(w_{11}, w_{12}, \dots, w_{18})$ 을 만족하는

$w_{11}, w_{12}, w_{13}, w_{14}, w_{15}, w_{16}, w_{17}, w_{18}$ 의 값들을 구하는 것이다.

$$w_0 = (w_{01}, w_{02}, \dots, w_{08}), \quad w_1 = (w_{11}, w_{12}, \dots, w_{18})$$

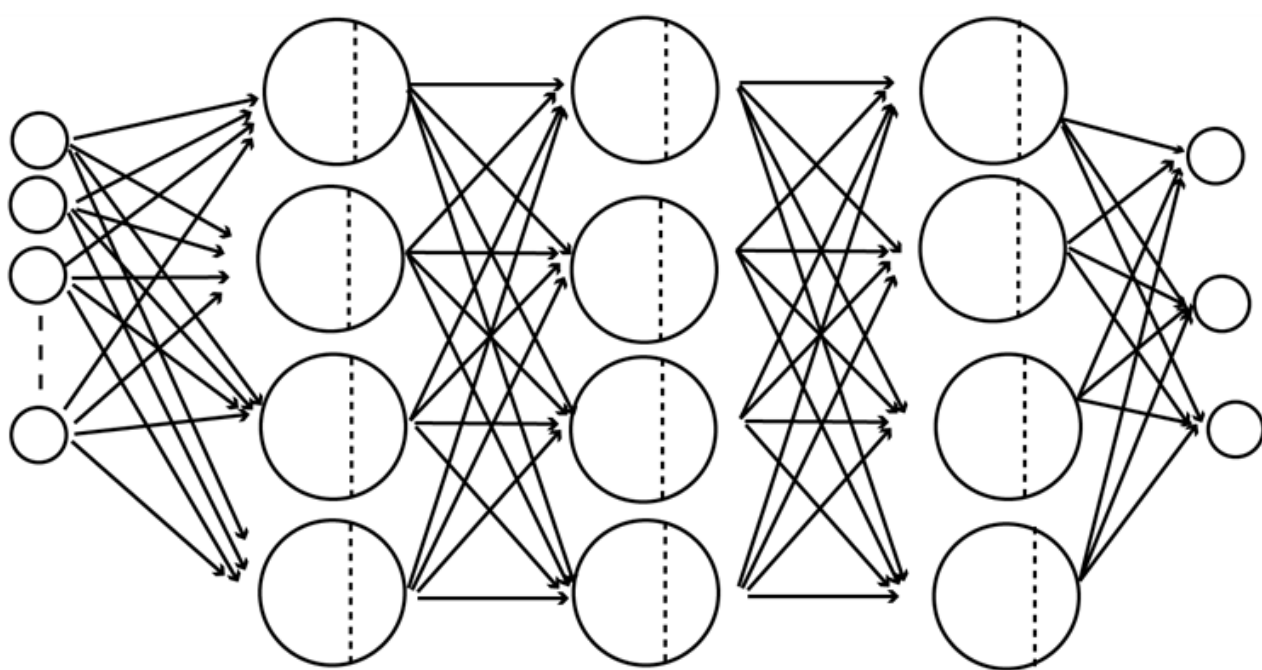
$$w_1 = w_0 - \alpha \nabla E(w_0), \quad \text{여기서 } 0 < \alpha < 1 \text{로 주어짐}$$

$w_0$ 는 주어진 값이고  $\nabla E(w_0)$ 의 값만 구하면 된다.

$$\nabla E(w_0) = \left( \frac{\partial E(w_0)}{\partial w_1}, \frac{\partial E(w_0)}{\partial w_2}, \dots, \frac{\partial E(w_0)}{\partial w_8} \right)$$

이들 미분을 계산하는 것은 중간층, 특히 입력이 가까운 깊은 층의 파라미터일수로 까다로워진다고 알려져 있다. 사실 적으로도 신경망의 층이 많아지면  $\nabla E(w_0)$ 의 편미분 값을 동시에 구하는 게 매우 복잡하다.

거기다가 실제로 프로그래밍하기도 어렵고 동시에 계산비용도 크게 증가 한다고 한다. 한마디로  $\nabla E(w_0)$ 을 구하는 게 어렵다고 알려져 있다.





## 예제2: 역전파 알고리즘

머신러닝을 위한  
선형대수학

12 신경망의 기본 구조와  
행렬 연산

3. 역전파 알고리즘

$$\text{업데이트: } w_i^+ = w_i - \alpha \times \frac{\partial E}{\partial w_i}$$

**Step 3:**  $w_5, w_6, w_7, w_8$  업데이트하기

(3.1)  $w_5$  업데이트

먼저  $\frac{\partial E}{\partial w_5}$  을 chain rule을 사용해서 구해보자.

$$o_1 = w_5 \cdot outh_1 + w_6 \cdot outh_2,$$

$$outo_1 = \sigma(o_1) = \frac{1}{1 + e^{-o_1}}$$

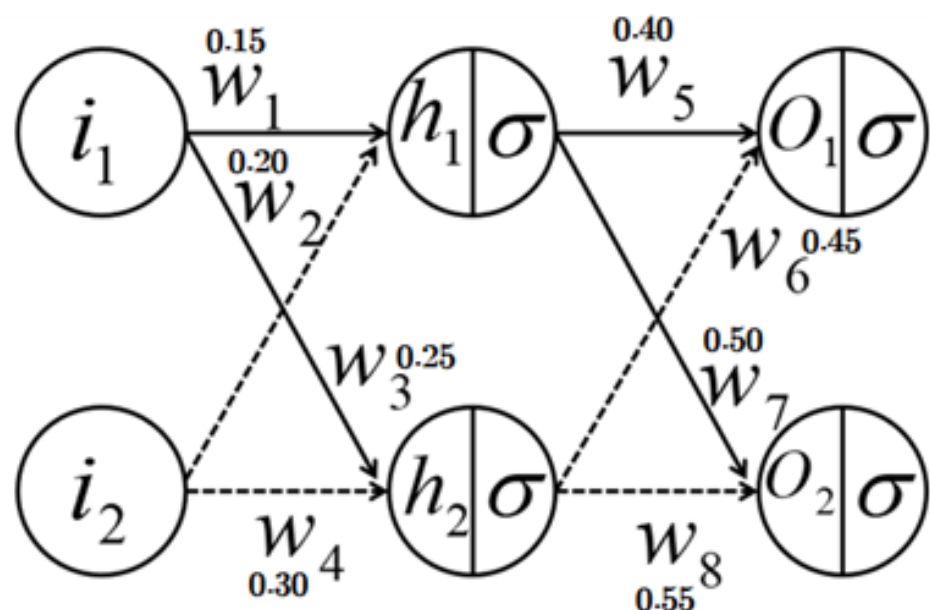
$$E_1 = \frac{1}{2} (result_1 - outo_1)^2$$

$$o_2 = w_7 \cdot outh_1 + w_8 \cdot outh_2$$

$$outo_2 = \sigma(o_2) = \frac{1}{1 + e^{-o_2}}$$

$$E_2 = \frac{1}{2} (result_2 - outo_2)^2$$

$$E = E_1 + E_2$$





## 예제2: 역전파 알고리즘

머신러닝을 위한  
선형대수학

12 신경망의 기본 구조와  
행렬 연산

3. 역전파 알고리즘

$$\frac{\partial E}{\partial w_5} = \frac{\partial E_1}{\partial w_5} + \frac{\partial E_2}{\partial w_5} = \frac{\partial E_1}{\partial w_5} + 0 = \frac{\partial E_1}{\partial w_5}$$

$$\frac{\partial E}{\partial w_5} = \frac{\partial E_1}{\partial outo_1} \cdot \frac{\partial outo_1}{\partial o_1} \cdot \frac{\partial o_1}{\partial w_5} \text{ 구하기}$$

$$\frac{\partial E_1}{\partial outo_1} = outo_1 - result_1 = 0.6065 - 0.01 = \mathbf{0.5965}$$

$$outo_1 = \sigma(o_1) = \frac{1}{1 + e^{-o_1}}$$

$$\frac{\partial outo_1}{\partial o_1} = outo_1(1 - outo_1) = 0.6065(1 - 0.6065) = \mathbf{0.2387}$$

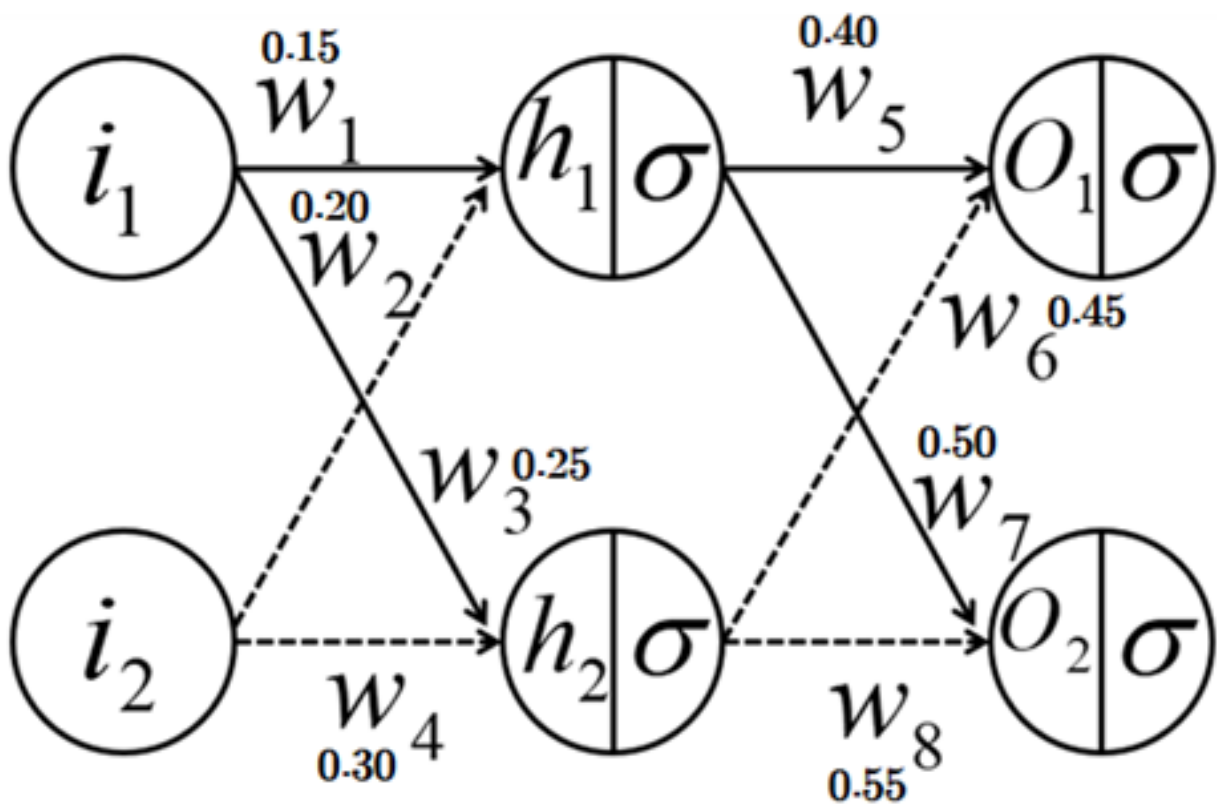
$$o_1 = w_5 \cdot outh_1 + w_6 \cdot outh_2$$

$$\frac{\partial o_1}{\partial w_5} = outh_1 = \mathbf{0.5069}$$

따라서

$$\frac{\partial E}{\partial w_5} = \frac{\partial E_1}{\partial outo_1} \cdot \frac{\partial outo_1}{\partial o_1} \cdot \frac{\partial o_1}{\partial w_5} = 0.5969 * 0.2387 * 0.5069 = \mathbf{0.0722}$$

$$w_5^+ = w_5 - \alpha \times \frac{\partial E}{\partial w_5} = 0.4 - 0.5 * 0.0722 = \mathbf{0.3639}. \quad \alpha = 0.5$$





### (3.2) $w_6$ 업데이트하기

$$\frac{\partial E}{\partial w_6} = \frac{\partial E_1}{\partial outo_1} \cdot \frac{\partial outo_1}{\partial o_1} \cdot \frac{\partial o_1}{\partial w_6} \text{ 구하기}$$

$$E = E_1 + E_2 = \frac{1}{2}(outo_1 - result_1)^2 + \frac{1}{2}(outo_2 - result_2)^2$$

$$\frac{\partial E_1}{\partial outo_1} = outo_1 - result_1$$

$$outo_1 = \sigma(o_1) = \frac{1}{1 + e^{-o_1}}$$

$$\frac{\partial outo_1}{\partial o_1} = outo_1(1 - outo_1)$$

$$outo_1 = \sigma(o_1) = \frac{1}{1 + e^{-o_1}}$$

$$\frac{\partial outo_1}{\partial o_1} = outo_1(1 - outo_1)$$

$$o_1 = w_5 \cdot outh_1 + w_6 \cdot outh_2$$

$$\frac{\partial o_1}{\partial w_6} = outh_2$$

$$w_6^+ = w_6 - \alpha \times \frac{\partial E}{\partial w_6}$$



## 예제2: 역전파 알고리즘

머신러닝을 위한  
선형대수학

12 신경망의 기본 구조와  
행렬 연산

3. 역전파 알고리즘

(3.3)  $w_7$ 업데이트하기

$$\frac{\partial E}{\partial w_7} = \frac{\partial E_2}{\partial outo_2} \cdot \frac{\partial outo_2}{\partial o_2} \cdot \frac{\partial o_2}{\partial w_7} \text{ 구하기}$$

$$E = E_1 + E_2 = \frac{1}{2}(outo_1 - result_1)^2 + \frac{1}{2}(outo_2 - result_2)^2$$

$$\frac{\partial E_2}{\partial outo_2} = outo_2 - result_2$$

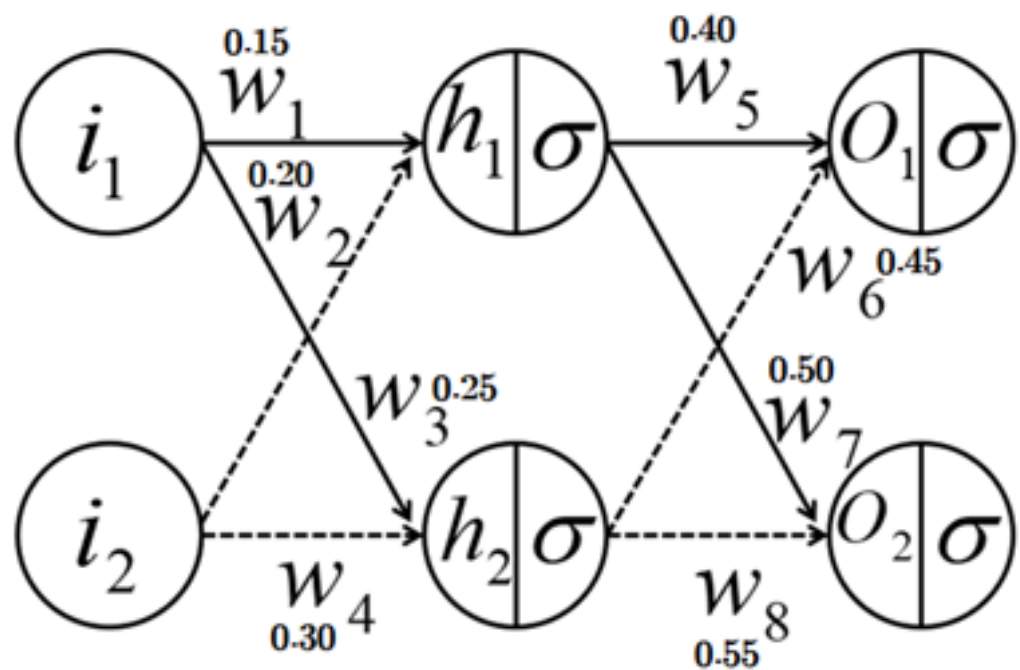
$$outo_2 = \sigma(o_2) = \frac{1}{1 + e^{-o_2}}$$

$$\frac{\partial outo_2}{\partial o_2} = outo_2(1 - outo_2)$$

$$o_2 = w_7 \cdot outh_1 + w_8 \cdot outh_2$$

$$\frac{\partial o_2}{\partial w_7} = outh_1$$

$$w_7^+ = w_7 - \alpha \times \frac{\partial E}{\partial w_7}$$





## 예제2: 역전파 알고리즘

머신러닝을 위한  
선형대수학

12 신경망의 기본 구조와  
행렬 연산

3. 역전파 알고리즘

(3.3)  $w_7$  업데이트하기

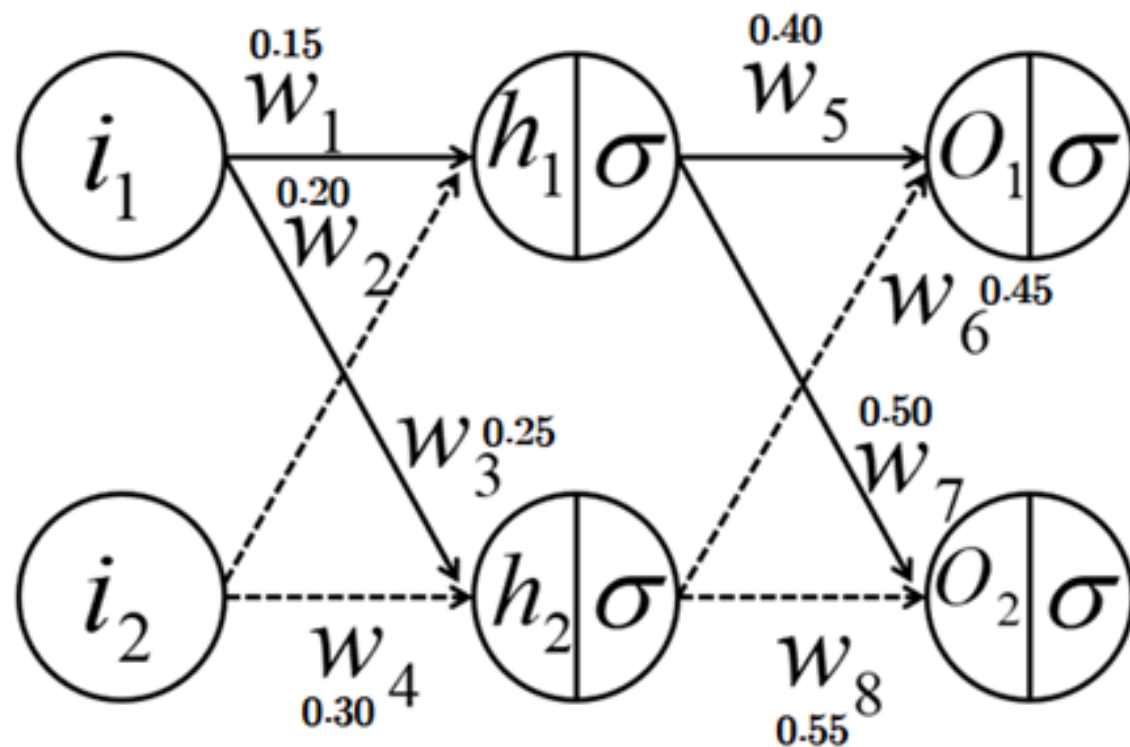
$$o_2 = w_7 \cdot outh_1 + w_8 \cdot outh_2$$

$$outo_2 = \sigma(o_2) = \frac{1}{1 + e^{-o_2}}$$

$$E_2 = \frac{1}{2} (result_2 - outo_2)^2$$

$$E = E_1 + E_2$$

$$\frac{\partial E}{\partial w_7} = \frac{\partial E_1}{\partial w_7} + \frac{\partial E_2}{\partial w_7} = 0 + \frac{\partial E_2}{\partial w_7} = \frac{\partial E_2}{\partial w_7}$$





## 예제2: 역전파 알고리즘

머신러닝을 위한  
선형대수학

12 신경망의 기본 구조와  
행렬 연산

3. 역전파 알고리즘

$$\frac{\partial E}{\partial w_7} = \frac{\partial E_2}{\partial outo_2} \cdot \frac{\partial outo_2}{\partial o_2} \cdot \frac{\partial o_2}{\partial w_7} \text{ 구하기}$$

$$\frac{\partial E_2}{\partial outo_2} = outo_2 - result_2$$

$$outo_2 = \sigma(o_2) = \frac{1}{1 + e^{-o_2}}$$

$$\frac{\partial outo_2}{\partial o_2} = outo_2(1 - outo_2)$$

$$o_2 = w_7 \cdot outh_1 + w_8 \cdot outh_2$$

$$\frac{\partial o_2}{\partial w_7} = outh_1$$

$$w_7^+ = w_7 - \alpha \times \frac{\partial E}{\partial w_7}$$



**Step 4:**  $w_1, w_2, w_3, w_4$  업데이트하기

(4.1)  $w_1$  업데이트하기

$$w_1^+ = w_1 - \alpha \times \frac{\partial E}{\partial w_1},$$

$\frac{\partial E}{\partial w_1}$ 을 구하기 위해 체인공식을 사용한다. 이를 위하여  $w_1$ 이 합성되는 경로를 살펴보자.

$$h_1 = w_1 \cdot i_1 + w_2 \cdot i_2$$

$$outh_1 = \sigma(h_1) = \frac{1}{1 + e^{-h_1}}$$

$$o_1 = w_5 \cdot outh_1 + w_6 \cdot outh_2, \quad outo_1 = \sigma(o_1) = \frac{1}{1 + e^{-o_1}}$$

$$E_1 = \frac{1}{2} (result_1 - outo_1)^2$$

$$o_2 = w_7 \cdot outh_1 + w_8 \cdot outh_2$$

$$outo_2 = \sigma(o_2) = \frac{1}{1 + e^{-o_2}}$$

$$E_2 = \frac{1}{2} (result_2 - outo_2)^2$$



$$E_2 = \frac{1}{2}(\text{result}_2 - \text{outo}_2)^2$$

$$E = E_1 + E_2$$

$$\frac{\partial E}{\partial w_1} = \frac{\partial E_1}{\partial w_1} + \frac{\partial E_2}{\partial w_1}$$

$$\frac{\partial E_1}{\partial w_1} = \frac{\partial E_1}{\partial \text{outo}_1} \cdot \frac{\partial \text{outo}_1}{\partial o_1} \cdot \frac{\partial o_1}{\partial \text{outh}_1} \cdot \frac{\partial \text{outh}_1}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1}$$

$$\frac{\partial E_2}{\partial w_1} = \frac{\partial E_2}{\partial \text{outo}_2} \cdot \frac{\partial \text{outo}_2}{\partial o_2} \cdot \frac{\partial o_2}{\partial \text{outh}_1} \cdot \frac{\partial \text{outh}_1}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1}$$

(4.1)  $w_2, w_3, w_4$ 도 같은 방법으로 업데이트하기



초기:  $E_{total} = 0.3240$

E\_total for k = 1: 0.32395

E\_total for k = 2: 0.32295

E\_total for k = 3: 0.32195

E\_total for k = 4: 0.32096

E\_total for k = 5: 0.31997

E\_total for k = 6: 0.31899

E\_total for k = 7: 0.31802

E\_total for k = 8: 0.31706

E\_total for k = 9: 0.3161

E\_total for k = 10: 0.31515

E\_total for k = 90: 0.27769

E\_total for k = 91: 0.27762

E\_total for k = 92: 0.27756

E\_total for k = 93: 0.27751

E\_total for k = 94: 0.27746

E\_total for k = 95: 0.27742

E\_total for k = 96: 0.27738

E\_total for k = 97: 0.27735

E\_total for k = 98: 0.27732

E\_total for k = 99: 0.2773

E\_total for k = 100: 0.27728